

UNIVERSITÀ DI PISA



Department of Computer Science

Master Degree Thesis

Fast Access through Lightweight Block-Based Compression Algorithms

Author

Massimiliano Baglioni

Supervisor

Rossano Venturini

Academic year 2024/2025

Contents

1	Introduction	1
2	Current Technologies	3
2.1	Row Stores	3
2.2	Column Stores	4
2.3	Hybrid Approach	7
3	Motivations	8
4	Row Store implementation	10
4.1	Data layout	10
4.2	Read and write operations	12
4.2.1	Write	12
4.2.2	Read	12
4.3	Prefetching	13
5	Compression	13
5.1	Bit-packing implementation	14
5.2	Iterator and monotonic accessing	18
5.3	Adapting the implementation for index integration	20
5.3.1	Compression for index data types	20
5.3.2	Decompression and values reconstruction	23
5.4	Motivation for bit packing	25
6	Hybrid index implementation	26
6.1	Schema Definition	26
6.1.1	Schema fields	27
6.2	Schema file parsing in Rust	30
6.3	Index construction	33
6.4	Hybrid index structure	35
6.5	Populating the index	38
6.6	Index compression	41
6.7	Reading from the Hybrid Index	44
6.8	Querying the index	46
6.9	Queries adaptation	48

7	Benchmarking	50
7.1	Benchmarking process and used tools	50
7.1.1	Index benchmarking procedure and tools	51
7.1.2	Compression benchmarks procedure and tools	55
7.2	Compression benchmarks	56
7.3	Bit packing benchmarking against existing random access alternatives	59
7.4	The dataset	62
7.4.1	Impact of Bit Packing on memory usage	66
7.4.2	Monotonic access	68
7.4.3	Full Vector decompression performance	72
7.4.4	Accessing all indexes sequentially	73
7.5	Impact of Bit Packing on Cache Performance	74
7.5.1	Complete random access	78
7.6	Additional compression datasets	81
7.7	Benchmarking Hybrid Index on Real-World Datasets	85
7.7.1	Chicago Crime Dataset	85
7.7.2	Results on the dataset	88
7.7.3	Columnar oriented accessing and caching benefits	91
8	Conclusions	95

1 Introduction

This thesis investigates how lightweight block-based compression, and compression in general, can be leveraged to improve random access on sequences of integers.

In particular, the concepts were applied in the context of forward indexes, which perfectly fit the use case.

Different types of random access were required for the implementation, including complete random access, monotonic access, full traversal, and full decompression of the encoded vector.

Each required access has its own real-world use case, listed here in order of relevance:

- Monotonic access, this type of access consists of queries sorted in an ascending fashion, going from the smallest to the largest index queried. There are several use cases for this type of access, the most relevant is the retrieval of certain documents, coming from queries resolved on inverted indexes. Since inverted indexes are usually sorted, the ids queried in the forward index will be ascending and spaced between them, based on the matches that result from the inverted index query.

This type of access was the most important for the current implementation, which is why it achieves the fastest access time among all the others, as the developed data structures were specifically designed to optimize it as much as possible.

- Full traversal, this type of access is a special case of the monotonic access, where there is no spacing between the queried indexes, which is the densest the queries can get.

This type of access has different uses in real world scenarios, like zone indexes and analytical queries, where large subsets of columns are required to be available at once.

- Full decompression, this type of access completely decompresses the whole vector. Some algorithms cannot be adapted to work with compressed values, or do not accept compressed versions of them. Also in some cases it can be beneficial having the whole vector decompressed rather than its compressed version.
- Random access, classical random access, which allows to inspect target information regarding specific document ids in the forward index. This is the slowest among the different accesses, since it is the one that requires the highest flexibility which also implies less opportunities for optimizations and natural trade-offs in terms of access time.

In particular monotonic access was the one required to perform best. This drove to a specific implementation to maximize access speed with queries with increasing accesses.

These accesses will be later discussed again and more thoroughly, along with tests on real world datasets to show the performance achieved by both the compression and index implementation.

Compression was included in this work because the main focus was on reducing memory consumption and improving access times.

Compression can reduce access time when conditions are favorable, by lowering the memory bandwidth required during transfers and by allowing the CPU to be utilized more efficiently, without idling while waiting for memory accesses. The other trivial reason to use compression is obviously the reduction of the overall memory consumption.

Compression alone does not demonstrate a practical real-world benefit. To illustrate its effectiveness and provide support for a usable random-access implementation, a forward index was developed.

The main use case for this kind of index are search engines that need fast random access on integers. It was indeed tested on top of a set of queries provided by Ebay in order to prove a real use case for it.

The greatest benefits of compression are observed in columnar storage, due to the way data is organized and the typical characteristics of column-oriented datasets.

The implemented index is unique in that it combines both column and row storage in a single structure. This hybrid design was motivated by the lack of suitable alternatives in the current Rust ecosystem. Existing solutions either did not support both storage types or, when they did, operated at a level of abstraction too high delivering full fledged services rather than more flexible crates.

A significant portion of this thesis is dedicated to the index and the design choices behind its implementation, as it embodies the core concepts that form the foundation of this work regarding compression and its advantages for random access.

The thesis will present row storage and column-oriented storage, in order to clarify the motivations behind adopting a hybrid implementation. It will then discuss how compression is implemented and what types of compression fit best random access for real world queries.

The final section of the thesis will show some benchmarks and compare different other available compression schemes and the advantages gained from compressed column store compared to uncompressed row based storage.

2 Current Technologies

Nowadays there are multiple approaches for what concerns data storage and retrieval, each coming with its advantages and drawbacks. This Section will cover the current state of most popular technologies and approaches when it comes to store tabular data. This Section is meant to present effective strategies and justify the specific implementation made and what the advantages and drawbacks that come with it are.

Moreover, many concepts used to implement the hybrid approach are based on well-established technologies that have demonstrated their value and effectiveness.

For each approach presented in the Section: advantages, disadvantages, use cases scenario and real world applications will be presented in order to give a structure to compare the different technologies and their specific field of application.

2.1 Row Stores

Traditional databases are based on this approach, which stores table records as a sequence of rows [8]. This approach leads to have records stored entirely in contiguous space, and then process queries in a row oriented manner. This type of layout makes full record accesses fast, since all the information about the searched record is contiguous.

Row stores are the standard choice for most business applications, this because they are particularly suited for OLTP (Online Transaction Processing). A small digression to describe OLTP systems is needed as it helps understand what strengths of row stores are, compared to other technologies. According to IBM [14], OLTP systems tend to:

1. Process a large number of relatively simple transactions, which tend to be insertions, updates and data deletions. Simple data queries are also executed.
2. Enable multi-user access to the same data and ensure data integrity. Data integrity guarantees that two users cannot change the data at the same time, which means that transactions are executed following a certain order, avoiding inconsistencies when multiple users try to access the same data. An intuitive example can be a room reservation system, where two users try to book the same room, data integrity guarantees that only one of the two users is able to book the room instead of having both reserve the same room, which is clearly an issue.
3. Provide indexed data sets, this type of indexes are used for rapid search, retrieval and querying.

4. Be always available, any downtime in this type of systems can be a serious problem, so they tend to be always available, without data loss or much downtime. This type of system tend to do complete backups frequently.

OLTP systems are frequently compared to OLAP systems which are another type of systems that will be discussed later again. OLAP systems are used more for analytical queries and metrics and are less suited systems for row stores type of workloads.

Row stores are well suited for OLTP systems given their good performance on operations that involve single records, or those operations that require entire records.

Row stores have a significant advantage, compared to other technologies, on insert heavy workloads, where entire tuples are inserted inside the database. The row layout allows to write the information fast and give a significant advantage, compared to scattered writes that happen in column store, where each column value is written on a separated and potentially far memory location.

Naturally row stores have drawbacks as well which come from their storage layout like their advantages do. The biggest drawback comes from the fact that row stores retrieve the whole row when queried for a subset of the record's fields. The worst scenario happens when only a small subset of the record is needed, in which case the whole row is extracted even if only a small subset of it is required by the query.

To extract the meaningful information *projection* is applied and the requested value is returned. Projection is the process of extracting only the required fields from the row. This process reduces main memory consumption, but does not solve the extra disk bandwidth usage. Another issue that arises with column stores is that elements within a single column are stored far from each other because entire rows separate them. This scattering disrupts spatial locality along the column.

These drawbacks have been identified in OLAP optimization techniques that require operations based on columns and similar analytical queries that require accesses to single fields of the whole tuples.

Typical use cases for row stores are all traditional databases systems. For example MySQL [18], OracleDb [19] and PostgreSQL [21] all use this data layout for their low level data representation.

2.2 Column Stores

Column stores are another strategy used to store information compared to row stores. As the name suggests, column stores store data according to

columns rather than rows. This means that columns are stored in a sequential order on the storage medium. Each column is followed by the next and so on [36].

This data management makes column stores basically the opposite to row stores in terms of memory layout. The main idea behind this layout is to solve the issue that was discussed for row stores: avoid to fetch the whole row when only a subset of columns of the row is required.

Several advantages come from the column data layout, for example: accessing a single value of a row becomes faster, since data from a column is stored contiguously and no other values are retrieved during the query process.

Having column values close to each other give multiple benefits, for example if one value from a column is queried from different rows, this layout enhances spatial locality for this type of access. Enhanced space locality often leads to higher cache hits.

There are advantages also in terms of CPU operations, it is indeed possible to use SIMD operations to extract and work on multiple column values at the same time. This layout also allows room for parallelization strategies, given that the single column can be accessed by multiple processes together. Adjacent data within a column is often similar, and identical values may appear multiple times. This property is particularly useful in most compression schemes that can leverage this information and exhibit substantial optimization effects. Two examples can be bit packing strategies and dictionary based strategies, which both benefit from similar data characteristic. Column compression has been shown to speed up query execution [1], but only when using lightweight compression schemes. It is important to have a lightweight compression scheme since decompression time could outweigh the I/O savings, leading to a slower execution. This particular property that column stores have with compression, was used in the hybrid implementation presented in this work. In particular bit packing was used as a compression scheme, along with the use of SIMD operations to take advantage of modern processors capabilities.

These properties make column stores work better in OLAP workloads, which are usually read-mostly.

OLAP workloads, as IBM [13] states, are those workloads related to analytical processing on large volumes of data. Queries that are executed are mostly analytical and used to create reports and get information about the data, rather than satisfy user transactions. This also means that most of the work is reading the data rather than writing it, which is one of the biggest differences between OLAP and OLTP workloads.

The best use cases for column stores then are:

- Data warehouses: which are databases designed for efficient querying and analysis. Most involve OLAP queries, which perform aggregations over only a subset of columns, but across a large number of rows.

- RDF and graph: column stores are used to store RDF data, where a wide table of information is vertically partitioned into multiple two column tables, one for each predicate that can be later used inside queries.
- Search engines: search engines and decision support systems, which are the main scenario of OLAP workloads, serve a similar purpose. For instance MonetDB is often considered as a backend for search engine.

Column stores bring a lot of advantages if used in the correct scenario, but there are drawbacks that must be mentioned[1], [6], [8], [12], [24].

Column stores, different than row stores, struggle on write operations which, given the layout, become expensive. This high cost, comes from how tuples have to be inserted inside the database. Columns are scattered across the disk, data inside of them is sequential, but columns that come from the same entity are all stored in different disk locations, leading to high seek time and low locality. In row store this kind of operations are faster because the whole tuple is just written to memory immediately at the end of the current stored data.

Compression and dense packing make tuple movements within a page challenging, because moving a compressed entry may require decompression of the whole, or most, of the column.

Another disadvantage is that queries with long projection lists, or non selective predicates, tend to perform slower. The extra cost, in this case, comes from the tuple reconstruction that must be made after data fetching. Column stores fetch data first and then aggregate it to a resulting tuple that is returned to the query, this procedure takes longer for larger tuples.

Column stores are usually used and implemented in main memory, this makes them struggle in constrained environment that lack the required memory that is also non scalable.

Another situation in which column stores perform worse compared to row stores is when tuples fit in a cache line. In such cases row stores have the advantage of not having to reconstruct the tuple, avoiding a major overhead that column stores cannot avoid.

Column stores can be implemented from scratch, using a native approach like most commercial implementations do, or by adapting a row store to the column store strategies.

There are several ways to emulate a column store using a row store [3]. The first approach is to partition large tables into single tables which then represent a single column of the starting row.

Another approach is the creation of indexes on subsets of columns and access these indexes rather than accessing the entire table. By using this strategy the query optimizer can then avoid touching the base table entirely.

The last method presented uses views for frequently accessed columns. Materialized views cache results from queries so that next executed queries do not need to retrieve the whole data again, leading to make them faster query execution and reduced disk utilization. This approach is faster than the other presented, but comes with significant data duplication, since each view requires to copy data that is already present in the database. Overall all of these solutions are usually slower compared to native implementations such as C-Store.

There are several real world examples for column stores, for example MonetDB [17] and C-Store [28]. Both of these projects are native implementations of column stores and do not use the previously mentioned strategies to adapt row stores, but instead implement the column store natively.

2.3 Hybrid Approach

Both row stores and column stores bring advantages and drawbacks and work best in certain scenarios. In order to get the best out of both approaches, another paradigm can be used, which is a hybrid approach where both row and columnar are used. The goal is to implement both systems into one and use them where they work best.

This type of mixed workloads are growing in real world services where both user transactions and analytical insights are required. This type of mixed workloads emphasizes the importance of a system that can perform best in both scenarios.

There are different ways to create such systems. As already stated, an approach can be by using only a row store and then implement the column layout only for those columns that are involved in OLAP workloads. This approach leads to one row store based system that adapts to both workloads. This solution inherits the inefficiencies that non native column stores bring. To solve these inefficiencies another solution exists which requires to run two completely separated systems optimized for different workloads.

In this scenario the row store based system takes care of OLTP transactions, which includes updating the data based on the queries made by the users. Data analysis is carried out by another independent system based on column stores. This other system is another database that duplicates the data inside the row based one.

This type of implementation benefits from the writing capabilities of row systems and from the analytical capabilities offered by columnar systems that can leverage all the optimizations that are exclusive to native columnar stores.

Such a system introduces complexity in terms of infrastructure and adds data duplication, the data that is stored inside the columnar database is identical to the one stored in the row based system.

On top of this the column store needs to be updated periodically since it is updated in batch, given its lack of writing capabilities it is more efficient to issue big updates rather than keeping up with the small ones that are executed on the row one.

Analysis usually requires recent and fresh data, which in turn requires these updates to be frequent, otherwise data becomes outdated and not relevant to the current state of the database. Analysis can be executed during low transaction time, where the database can have the latest data available without much change happening during the analysis computations.

There are different real world examples of both commercial products and research projects which prove that such systems are worth developing and exploring. As it can be seen in their documentations [5] [27], both StarRocks and Apache Doris allow to use a hybrid approach. These products allow to explicitly choose, for each table, how it is stored. If the hybrid approach is chosen, then data is duplicated and both row and column store is used.

Another example of commercial use of the hybrid approach is Teradata, which is also backed up by research paper [4] that show its structure.

Another example is Hyrise [11], which is an in-memory database system open source that is research oriented and not a commercial product.

All of these examples motivated the work to use such a hybrid approach, but at a lower level compared to these products. In fact the aim was at the library level rather than a complete system. The idea was to implement an interface to handle the memory layout that allows to build on top of it in order to then implement a system or a tool that uses these concepts but at a low level.

3 Motivations

The project was designed to combine the advantages of both columnar and row storage, and to exploit compression on the columnar portion in order to increase access speed and reduce memory consumption.

As mentioned in the previous Section 2, various tools, both commercial and open-source, already exist. It is widely accepted that one should avoid reinventing the wheel whenever possible, to prevent wasting time developing a solution that may ultimately be inadequate. This was not the case though, through extensive research, no software nor tool was found that satisfied the requirements that the project had.

Different database systems provide hybrid storage, but they all operate at a high level. The aim of this project was to work at a lower level, developing a crate that allows building such database systems on top of it, rather than delivering a full system out of the box. This low-level approach also enables the implementation of lightweight compression on the columnar portion and

makes the system extensible, allowing additional compression schemes to be integrated in the future. There is currently no such solution in the Rust ecosystem, which is the main justification for developing this library.

Different solutions already exist for row storing in Rust, at both the crate level and at a higher level, so there was no need for another. The same goes for column storage, where different crates are already available like Tantivy [23] and Strawboat [29].

Tantivy is a good comparison since it gets close to what we propose in this work. The main difference with Tantivy is that our contribution aims to store different type of data of multiple types, integers, floats, strings and so on, while Tantivy focuses on being the foundation for a text search engine, so it makes use of inverted indexes for string storage and only allows for columnar stores, via **fast fields** for other types.

Tantivy allows for both row and column storage for fields defined in the schema, using **fast fields** for columns and marking them as **stored** when kept as rows.

Tantivy’s row store though is not meant for random access and for intensive search on it, directly from the documentation [23]:

“It is NOT meant to be called for every document matching a query. As a rule of thumb, if you hit the docstore more than 100 times per search query, you are probably misusing Tantivy.”

Tantivy’s row store relies on a general-purpose compression algorithm like LZ4, which makes random access queries relatively slow. In contrast, our implementation is designed to support fast random access on both columnar and row stores, while applying lightweight compression specifically to the columnar portion to avoid slowdowns on compressed data access.

Tantivy allows for column store though, as previously mentioned, using **fast fields**. This type of fields are used to do aggregations, but they also allow for random access, that has fast performance.

The storage of these fields uses the concept of segments, which, again, differs from our implementation. Different fields are stored in different segments, which are pieces of the index. Having them divided avoids rewriting the whole index each time an operation is performed, with this approach new documents, that are added during execution, are added to the segment that is later merged with the overall index. The use of these segments makes docs ids relative to the segments though, which, if querying documents using their absolute id is needed, requires to either map segments and ids or merge segments frequently in order to have only one when querying.

All of this is adopted for performance and scalability reasons, but makes random access quite less immediate and this is something that is addressed in our implementation. On top of that Tantivy’s column store, at the time

of writing, supports only `u64`, `i64`, `f64`, `ip`, bytes and text for its column store.

To summarize, the motivations for this work came from the lack of a low-level solution in Rust that allows defining the memory layout of stored data and applying lightweight compression to optimize both access speed and memory usage.

Some alternatives exist but none of them completely addresses the need of having both columnar and row store available in the same crate, giving the flexibility to chose between the two storage paradigms allowing for random access and compression.

4 Row Store implementation

My implementation was developed on top of the row-only index previously created by Professor Rossano Venturini [33], extending it with additional functionality. Describing its structure is crucial since the overall contribution is based on expanding and adapting this base implementation to achieve the required goals.

The following sections describe this implementation, which will also be cited in subsequent sections.

4.1 Data layout

The overall index is represented as a structure that contains a `len` field representing the number of rows in the index, a `DenseRowDescriptor` to represent what types a row contains, a vector of `u8` called rows used to represent the actual data in memory and another vector of `u8` called blobs to represent Binary Large Objects and strings.

In the initial implementation there already was dedicated code to enforce typing before new rows insertion, in order to define a structure of the dataset that resembled the schema of the same dataset.

In this section, this portion of the implementation is described. The data structure was subsequently extended to support a strict schema definition, enabling a robust interface for instantiation according to the dataset's defined data types.

The first type that is used is the `DenseRowDescriptor` type, which defines the types that are present in each row of the dataset.

This struct type contains a vector of `FieldDescriptor`, which describes the columns of the index. The structure also keeps track of a permutation vector. This vector is used to keep track of the order of fields that which might change during the construction phase. A permutation is used in order to reduce the space used by Boolean types.

Based on how many fields are present in the index, a certain amount of bytes is prepended to each line, these bytes are called validity bytes. Validity bytes associate one bit to each field present in the index to keep track of the presence of the field in the row. This method is more efficient in some cases since it allows to access only the single bit associated with the specific field, rather than accessing the value at the field's offset in the row.

To further optimize the data structure, the permutation previously mentioned comes into place. Instead of using one byte for each Boolean, each Boolean is packed using a single bit. In order to do this though, all Booleans are stored in the first data byte together, operation that requires a change of the original fields' order.

To later access the fields, using the same initial order indexing, the permutation vector is used to match old fields positions to the current permuted positions in the row. Otherwise, knowing the new indexes would be required, which is clearly not as intuitive and error prone.

When storing Boolean values, one initial byte is reserved. If there are more than eight values, an additional byte is allocated, and this process is repeated until all values are stored. Any remaining bits are left as padding.

As mentioned, `DenseRowDescriptor` makes use of the `FieldDescriptor` type. This type contains all the information needed to find the field in the row during the access process.

Inside this struct type an id is stored and then used to identify the field itself.

The struct also stores a local id that is calculated based on the field id. This local id information is used to get what bit is used in the validity bytes to represent the field.

Another field used to find the field inside the row is the byte offset, this value represents where the actual data starts in the row. By the use of these two fields it is then possible to find the value of each field inside the rows.

The last field relevant to the data structure's data layout is the `bit_offset` field. This field is used exclusively for Boolean values and indicates the specific bit within the byte where the Boolean is stored, as discussed earlier. Using this field in combination with the byte offset it is possible to find Boolean values inside the rows.

`FieldDescriptor` contains an `f_type` field of type `FType`, which is a type used to describe the type of the field.

`FType` is a custom type that represents all the primitive data that can be stored in a row of the data structure. This custom type is needed since it can be used as a parameter for functions without compile time knowledge of the passed primitive type. This type's main function is to basically generalize functions, without the need of generics.

Along with `FType` also `GenericType` is defined. This other type differs from

FType by storing any primitive value that is represented by an **FType**. In this way, the type of the field and the actual value that is then stored inside of it can be separated with two types, where one brings concrete values as well.

4.2 Read and write operations

This Section will describe how read and write operations are performed in the index. These operations are important due to the fact that they are used even for the columnar and compressed version of the index.

4.2.1 Write

After the **DenseRowDescriptor** is defined for the index it can be used to write and read different rows. As previously mentioned, inside the row descriptor there's a vector that contains fields descriptors that have the necessary information to find where fields are stored inside rows.

The write function takes a **GenericType** that will hold the value that must be written in the data structure. If the passed value is **None** then the validity byte is set to zero and there is no need to store the value, which is another benefit coming from the use of validity bytes in this scenario.

If the value is not **None** and can be written, then the corresponding bit is set to one in the validity bite and the value is written to a row vector that is passed to the function. It is important to notice that this function does not write directly to the actual row vector of the data structure, but to a passed vector that is then used by the **add_row** function to extend the **rows** vector. The **add_row** function actually writes in the data structure and creates a row vector of length, equal to the row descriptor length each time that it is called. Before passing the value to the **write** function, they are permuted using the permutation vector.

It can be seen that this type of writing is clearly row based, each row is written together and added to the end of the **rows** vector.

4.2.2 Read

There are two ways to read data from the data structure: one is calling the **access** function, the other is calling the **get_row** function.

Both of the functions make use of the **read** function defined for the **FieldDescriptor** structure type. This function is used to calculate the correct values to call another function that is **read_scalar_value**, which is the function that actually returns the value read from the row. This last function takes the row and the offset in order to read the value, it makes use of a generic so that it can be called on every type that can be defined for rows.

The field descriptor’s read function calculates the offset, the validity byte value and the validity bit and then, based on the type of the value that is contained in the field descriptor, calls the `read_scalar_value` accordingly. The `access` function that is defined by the index structure is used to retrieve a single field from a specific row. The passed values to this function are indeed the `row_id` and the `field_id` in order to find the specific value. This function makes use of the previously defined read function implemented for the specific queried field, that is found by using its id.

The `get_row` function returns a whole row, starting from a given id just by calling the read function on each field of the row and collecting the results inside a vector which is later returned as a result.

4.3 Prefetching

Prefetching, a technique widely used in memory-intensive systems [32], was implemented to improve performance.

To implement it the `_MM_HINT_NTA` was used, from the rust documentation [26]:

“Fetch data using the non-temporal access (NTA) hint. It may be a place closer than main memory but outside of the cache hierarchy. This is used to reduce access latency without polluting the cache.”

The use of prefetching allows cache lines loads before a miss happens, improving the cache hit rate. In this specific implementation, since queries are predictable, or come in batches, prefetch can be used during the queries parsing process. In this scenario the program caches queries that are read before executing them, using the previously mentioned operation.

Prefetching was used not only in the row based index, but also on the hybrid implementation as it will be explained later.

5 Compression

It has been shown that compression can increase the query execution speed significantly for different reasons [2].

A major advantage of compression is the reduced memory footprint, which also leads to faster access times. By moving less data through memory and caches, bandwidth usage is reduced, resulting in improved overall performance.

The data size reduction also favors cache hit rate, since more significant data will fit inside the cache, taking less space, which basically increases the density of information that can be stored inside the cache.

Column stores, as presented earlier, suffer from seek times caused by data being scattered across the disk. Data compression can help keeping data

closer, since the space between columns is reduced by how compressed the data is.

Certain compression schemes are even more justified and beneficial for certain operations. It is possible to operate on the compressed data, completely avoiding the decompression overhead. For example if RLE (Run-length encoding) is used, a `SUM` operation can be used directly on the compressed data that encode the number of sequential identical numbers. This sort of operation are highly beneficial and reduce the CPU cycles count by a lot.

On top of this, compression is highly synergic with column stores, since similar data is stored close, leading to higher compression rates on certain compression schemes.

As already mentioned, fast compression scheme are preferred to multi purpose heavy compression schemes to avoid spending more time decompressing than actually fetching the data. The idea is that, since the CPU is much faster than the data transfer, it remains idle, waiting for data. Reducing the space not only allows to move less data and reduce these transfer times, but also makes the CPU work instead of being idle waiting for data transfer.

These concepts are the reason why the currently supported compression scheme is bit packing.

Bit packing is beneficial only under certain scenarios. As it will be described later, the current implementation uses some caching mechanism to exploit the specific monotonic accesses specified by the requirements, which leads to some overhead. This overhead can easily outweigh the compression gains coming from bit-packing encoding if not applied in favorable scenarios.

The current implementation supports bit-packing only, but it is built to work with any compressor that implements the `compressor` trait. This decision comes from the requirement of being able to chose between different compressors based on the characteristics of the data, in order to use the most beneficial compression scheme.

For example, a dataset with small numbers and a small difference in terms of value, would gain a lot from the use of bit packing, while a dataset with big numbers repeated in long sequences could benefit more from RLE. This is just a simple example to give the idea, but a lot of fast decompression algorithms can be beneficial and rule them out completely from the implementation would have made little sense.

5.1 Bit-packing implementation

The implementation is based on the famous `bitpacking` crate [9] which is based on Lemire's work [15]. The decision not to implement the whole compression procedure from scratch was made since the crate is highly optimized and probably the best solution for such a scenario and would probably be impossible to beat with its performance with a limited amount of time.

The crate can operate using different approaches that are distinguished by three different ways of how the bitpacker is instantiated, which are: `BitPacker1x`, `BitPacker4x` and `BitPacker8x`.

The difference stays in the instructions that are used for compression. The `1x` version does not use SIMD instruction and simply concatenates instructions during compression and decompression. The other two versions instead use SIMD instructions.

SIMD (Single Instruction, Multiple Data) instructions allow the CPU to perform the same operation on multiple data elements simultaneously, significantly accelerating data-parallel tasks.

Compression is a type of task that highly benefits from SIMD instructions. The difference between the `4x` and the `8x` version is that the first uses `SSE3` instructions while the second uses `AVX2` instructions. `SSE3` operates on 128-bit registers and offers limited enhancements over earlier SSE versions, mainly for floating-point operations. In contrast, `AVX2` uses 256-bit registers, supports both floating-point and integer operations, and provides greater throughput and efficiency.

Even though `AVX2` can be more efficient, for compatibility reason, the `4x` is adopted in the current implementation. It is possible to switch to the `8x` version with minimal changes. These changes involve mainly the number of elements per blocks that doubles, since both compress and decompress functions are able to take double the amount of numbers.

By changing a pair of constants in the code, it is possible to switch to the `AVX2` version. The code was designed with modularity in mind to maximize flexibility for scenarios such as this.

The crate, for efficiency reasons, compresses using blocks of fixed size. In particular the `4x` version compresses using blocks of 128 `u32`, this choice led to different implementation details that will now be covered.

The compression is handled by a struct that holds the compressed blocks and extra information that are needed to later decompress the data and to enhance the compression ratio of the data.

The overall compression is handled by dividing the input data as blocks of the defined fixed size. This approach is beneficial both for SIMD instructions computations and for compression ratio in certain scenarios.

About the compression ratio, if the dataset contains mostly small numbers, but still has a wide range (an high difference between the minimum and the maximum values found in the dataset) applying bit packing can still be beneficial.

Dividing the data in blocks allows to isolate large numbers, requiring an high number of bits, only for the block where it ends up being, instead of requiring the whole dataset to use such a number of bits.

This typically results in a higher compression ratio when the dataset is well suited for block-based compression algorithms.

This approach requires to chose blocks dimension, which also determines the storage overhead.

The block dimension must be balanced with the average number of bits used to represent numbers. The higher the block dimension, the higher impact a large number, among small ones, will have. The smaller the block dimension, the less impact a large number will have.

Small blocks, though, require more data to be stored, in order to allow later decompression. The dimension then must be calibrated in order to reduce the impact of sparse big numbers, keeping the extra information for decompression overhead small.

The current implementation uses a standard block size, for now, that matches the library requirement, either 128 or 256. It is still possible to adapt it to a variable block size.

Allowing variable block size gives more flexibility and allows changing the blocks size based on the dataset.

As already stated, for each block, it is mandatory to store the number of bits used to encode that block, in order to be able to decompress it later.

To hold these values, a prefix sum is stored inside the structure.

A prefix sum was preferred, since it allows the access to the number of bits used for a specific block with a time complexity of $\mathcal{O}(1)$. On top of that, when decompressing, the start index of the block and its end, in the compressed `u8` vector, is required. Gathering the start and the end of the block reduces to a multiplication using the prefix sum, since blocks have a fixed size.

Target block's start will be equal to the total amount of bits used by all previous blocks (which is the semantic of each element of the prefix sum) times the length of the block, divided by eight to get a bytes value. Also these operations end up being $\mathcal{O}(1)$. So as a formula:

$$\text{start byte}(i) = \left\lfloor \frac{S_i}{8} \right\rfloor, \quad \text{start bit}(i) = S_i \bmod 8$$

where $S_i = \sum_{j=0}^{i-1} L \cdot b_j$ is the total number of bits used by all previous blocks, L is the block length (number of values per block), and b_j is the bit-width of block j .

The overhead ends up being storing this prefix sum that is represented using a vector of `usize`, which is one `usize` per block.

The type of the prefix sum vector depends solely on the size of the dataset to compress and on the sizes of values to compress, a type that does not overflow is enough.

In our implementation blocks of 128, or 256 `u32` are used. Data is handled based on its size compared to the blocks size (for simplicity 128 is assumed), so if data to compress is:

1. Shorter than the block size, length less than 128, then zeroes are padded to fill the block, until the block size is reached.

2. Equal to the block size, data is immediately compressed.
3. Greater than the block size, it is split in blocks of the defined block size. If the last block is lower than the block size, the first approach is applied.

Whenever padding happens, a number is stored in the struct to keep track of how many zeroes have been padded to the block, in order to avoid inconsistent decompressed blocks. The previously mentioned prefix sum is populated during this process as well.

Whenever a block is compressed the compression result is a vector of `u8`, that is appended to a vector called `data` that stores the overall compressed vector.

This construction is applied inside the `from` function, which uses a generic parameter for the passed data to compress. The generic requires the data to implement the trait `AsRef` for vectors of `u32`, allowing the compressor to accept both vectors and references to vectors of `u32`.

Several methods are implemented to decompress or access compressed values. The `unpack` method decompresses the whole compressed array, decompressing one block after the other. This procedure is not often used, but could benefit from parallelization if the compressed data is big enough.

More interesting is the one block unpacking function. This function uses the prefix sum in order to find the start, the end and the number of bits used to encode the block to decompress.

This function is much more used than the previous since a single block is the smallest unit of decompression of this implementation.

It is not possible to decompress a single number, the whole block needs to be unpacked. This limitation brought up to the implementation of a caching system in order to avoid useless decompression of more than one block that will be discussed shortly.

Functions to query directly for a single value inside the compressed data are present. These functions start from the index of the element in the original vector and allow for random access. The procedure finds the block where the number is stored and unpacks it using the above mentioned `unpack_one_block`, then the position inside of the queried value inside the block is obtained by a modulo operator, using the block size, on the index in the original array.

After finding the position relative to the block the value is returned with a simple access to the decompressed block.

The restriction to block sizes of either 128 or 256 enabled further optimization. Whenever modulo or division operations are required, they are implemented using bit shifts, which are significantly faster than standard arithmetic operations. Because the block sizes are guaranteed to be powers of

two, shifts can be used in place of conventional operations. This results in faster decompression and access times by substantially reducing the number of CPU cycles required for each access.

5.2 Iterator and monotonic accessing

One requirement for the data structure was to have an efficient retrieval of values in a forward like access pattern.

This means that the data structure is required to be efficient when queried using random access in an ascending like access pattern, starting from an index and going only forward in its accesses. This behavior matches exactly what a forward iterator does.

In order to do this a custom iterator for the structure was implemented.

The main optimization that had to be implemented was regarding the decompression of blocks. Imagine this forward random access pattern on the data structure, based on the given set of indexes to be accessed, it can happen that multiple indexes are stored inside the same compressed block. If the data structure did not take advantage of this information, the retrieval of the two values would require the decompression of the same block twice. The optimization that was implemented for the iterator stores a decompressed block that will be called a cached block.

The cached block is initialized with the first block of the data structure decompressed. Whenever an iterator is created, it checks if the required value is inside the cached block, in that case the value is immediately returned with the exact cost of an array access.

Whenever the queried value is not inside the cached block, the corresponding block where the index resides, is found by doing a simple integer division by the `BLOCK_SIZE` and the value inside the block is found using a modulo operator (both implemented using bit shifts).

In this case the new block found is decompressed and cached by overwriting the previous block. The block is overwritten since the access should follow a forward approach, which means that the block is no longer accessed after it gets out of the query scope.

Clearly the more the query indexes are close, the better this optimization works considering that, for every access in the same block, one less complete block decompression is executed.

This optimization comes with the overhead of storing a complete block of `u32` decompressed, which is pretty reasonable in the specific use case.

The first implementation of this approach stored the cached block inside the iterator and an extra struct called `RandomAccessSupport` that was inside the compressor struct.

This architecture allowed the instantiation of more iterators at the same time, where each one stored one cached block.

This first solution though was not much clear about the separation of specific concerns between the different structs, since there was one wrapping struct that basically took care of every aspect of the procedure from the compression, to the decompression to the iteration.

To make things more maintainable, the code was refactored, to reach the current design, where the `BitPackedVec` is the struct that compresses the data, then it can be converted, using the appropriate `into_buffered_decompress` function, to a struct that allows for the efficient iteration, random access and decompression.

In the current design this structure is called `BufferedBitPacking` and it stores the cached block and handles all the decompression functions. The struct also allows for random access on any given value inside the vector implementing the `RandomAccess` trait.

This trait requires the implementation of the `get` and `get_unchecked` functions that, given the original index of the value, should return the value by decompressing it from the compressed vector.

The idea behind the trait is to group compressors that allow for efficient random access, in order to allow the user to choose the best to use based on the properties of the dataset.

Currently, only bit packing is implemented. This method suits random access well, since retrieving a single value within the compressed data requires decompressing only the corresponding block, rather than the entire vector. Like the iterator does, also the `get` functions exploit the cached block. The iterator just holds in memory an index of the last retrieved element and calls the `get` function on it. In the current implementation only one cached block is stored in the overall struct, before there was one `RandomAccessSupport` struct for each iterator and one for the decompression struct.

This meant that both the struct and the iterators had two `get` functions where each accessed to their own structs for random access. This was not clear implementation wise, so we have decided to switch to the current code structure which is much clearer and less convoluted.

Furthermore, some of the default implementations of iterator functions in Rust were not correct for our use case and required custom implementations. Among these, the most important is the `next` function, which must be explicitly overloaded.

Its implementation is relatively straightforward: it simply calls the `get` function, which either retrieves the value from the cached block if the index falls within it, or updates the block and then returns the requested value.

The issue though happened when the `skip` function was called in other sections of the program. During the analysis of the flamegraph [10] of one execution, a lot of time was spent for the `skip` call on the iterator. The issue lied in Rust's default implementation of the function.

Rust's default implementation of `skip` creates a new `Skip` iterator struct with `n` as a parameter, where `n` is the number of elements to skip. In order to

skip this elements whenever `next` is called, then `nth` function is called with `n` as a parameter.

The issue though comes from the default implementation of `nth`, which calls the `advance_by` function which is just a for loop that calls `next` `n` times and discards the results keeping side effects applied to the iterator structs.

This can be optimal in certain iterators, but in our specific case leads to inefficiencies that can disrupt performance.

For example, calling `skip` to skip directly to the last block of a big compressed vector, would lead to the decompression of all blocks of the vector. This happens because `advance_by` calls `next` on each element that precedes the last block, which reduces to the decompression of each single block.

The expected behavior is instead the decompression only of the last block of the vector.

To achieve this, the new implementation of `next`, just moves the internal field `index` to `+n`, and calls `next` on the new index. This leads to the decompression of the exact block and avoids useless computations generated by the default implementation.

The `nth` function implementation has been rewritten instead of the `skip` one since the standard library documentation for `skip` says:

“Rather than overriding this method directly, instead override the `nth` method.”

5.3 Adapting the implementation for index integration

The described bit packing components were implemented to work as a standalone library in the initial state of the project, but were later adapted to be used in the hybrid index implementation later.

This Section will explain changes and strategies used to adapt the generic bit packing crate to work on the specific data types used in the hybrid index.

5.3.1 Compression for index data types

The initial implementation, worked only with `u32` Rust’s primitive data type, which is too restrictive for the index scenario where multiple library defined data types are used to store data.

On top of this, the data stored by the index is not represented physically with its types, but as a vector of `u8`, which was not compatible with how the crate APIs work.

How the compression is applied inside the index will be later discussed with more context and more detail, but briefly presenting it now is necessary in order to understand how compression is applied to the passed data.

Indexes that are flagged as compressed, are not mutable, not in Rust’s sense, but in the sense that cannot be modified unless rebuilt.

In order to have the best efficiency possible, data is initially stored and muted in an uncompressed index, which is later compressed in one go.

This strategy avoids issues about expanding blocks, which requires to decompress multiple times and multiple blocks that can bring a lot of overhead. The one time compression strategy is reasonable as long as the index does not mutate often or does not mutate at all. Usually columns that are flagged to be compressed respect these requirements.

It is important to notice that these considerations hold for certain compression schemes and do not hold for others. Bit packing, implemented using blocks, does not suffer from updates of single values, since only the decompression of one block is required. Mid block updates can be problematic though when the inserted value exceeds the block's bit width. In such case all the subsequent data should be shifted by the now increased size of the block where the update happened.

It does not perform well on insertions in mid blocks, since it requires to decompress all successive blocks, which can bring a lot of overhead.

In our specific case the requirement allows us to assume immutable compressed indexes which brought us to this implementation choice.

As previously discussed, the main function introduced to adapt the current bit packing solution is `from_u8`. This function enables the compression of the already stored bytes of a given column using bit packing, returning a `BitPackedVec`, the compressed structure representing that column.

This function takes a generic `AsRef<u8>` which allows it to be flexible and accept both slices and vectors of `u8`, which in this particular case is the data physical representation as bytes.

The current implementation of the hybrid index, requires column stored fields to be physically stored by themselves. This means that the physical layout of column stores will store columns with one field only, it is not yet possible to store together multiple fields as a single column.

It was considered during the design phase to allow multiple fields aggregation and to store them together in a column-like layout, resembling small rows.

While this is not yet implemented, the current implementation was designed to be able to be easily adapted to support multiple fields per columnar block. Some functions like `from_u8` assume single fields per column and do not work if multiple are aggregated together.

As described earlier in Section 4 each row starts with some validity bytes and it is later followed by the real data.

This layout did not allow for the naive compression implementation that was used in the first place. What the first implementation did was to just compress `u8` data, considering them as `u32` which was inefficient both in terms of main memory usage and of compression ratio. The main memory usage was quadruplicated, since each `u8` was converted to a `u32` without

adding anything meaningful to it, just to allow compression with the native type.

The compression was completely off for different reasons. The validity byte with this approach, would be considered as data to compress. By considering it as real data to compress, the bit width used to encode it was the same of the real block, leading to a representation using the block bit width.

Since only one column is stored, the validity byte, in this case, will always use one and only one bit. Compressing it using the block bit width is a clear inefficiency that had to be solved.

Another issue arises when dealing with larger types composed of multiple `u8` values. For instance, compressing a `u32`, using this method, would require compressing each of its four bytes individually.

If, for example, only the least significant byte contains meaningful data, while the remaining bytes are zero, this approach would still store all thirty-two bits.

This results in a significant inefficiency, as only eight of those bits are actually needed, which is an overhead that is unacceptable in practice. This issue comes from the fact that each byte is considered as a stand alone value, while instead they are part of the same value.

This approach was discarded in favor of the current one that keeps track of validity bytes and of the logical representation of each value.

The `from_u8` function does not only take the data, but it also takes the `data_type` which is used to identify the logical representation of the stored values in memory.

The function initially chunks the data using the information about the data type size. The chunks dimension is equal to the data type size plus one because it takes account of the validity byte. This sum assumes that the validity byte is one only and that it contains only one relevant bit.

Each of these chunks will represent one value, the function separates the data values and the validity bit into two vectors in order to have a better compression ratio. The validity bit is extracted using a simple shift.

For values that are shorter or equal to `u32` the function takes a number of bytes according to their size. For example, for `u16`, two bytes are grouped and treated as a single `u32`. The function does this operation for each type and it is based on the data type that is passed as a parameter.

If the data type passed does not match the physical layout of the given data, then the resulting compression can have unexpected behaviors in terms of resulting values, since no checks are executed about it.

Values that are longer than `u32`, like `u64`, are compressed as two `u32`. This approach brings the same issues that the byte based approach brought, but in a smaller factor.

The scenarios where high inefficiencies arise should not happen frequently since bit packing would not be optimal, but other compression schemes should be preferred instead.

A solution to still use bit packing in this not favorable scenario with a high, but not optimal, compression ratio would be using delta encoding to reduce the number of bits used by the big numbers represented. The strategy would still use two `u32` increasing the compression ratio that still would not be optimal.

Validity bytes are separated from the actual data and get compressed as well. In this case the library is not used to compress the single bytes, but they are stored as bits directly, since each byte is considered to bring only bit of information. This means that eight values are packed in one byte and then stored in a dedicated vector for validity bytes which is later used during decompression.

The final value returned by this function is a struct that contains the validity packed byte vector, the resulting `u8` vector as fields and the data type as fields.

All these values are stored in the struct since they are all needed during the decompression phase that will be now described.

5.3.2 Decompression and values reconstruction

The main function implemented to support the index data types decompression is the `unpack_range_u8`. This function allows to use the same interface that is used in the standard row based index to access a single row.

The passed parameters are the start and the end of a sub slice of the vector storing bytes for the data. In the standard implementation these two parameters are the start of the row and the end is equivalent to `start + row_size`. In this way it is possible to read one row.

The major difference in the compressed version is that `start` and `end` do not coincide with the ones that are used in the standard version for two main reasons:

1. Validity bytes and actual data are stored in two different vectors in the compressed columnar version.
2. Data is compressed and its size is based on the number of bits used per block.

The first operation that is executed, in order to find the data inside the compressed vector, is to map the passed `start` and `end` to their values relative to the compressed data.

To calculate `real_start`, which is the start relative to the compressed data, the function subtracts to the passed `start` the value obtained by the division of `start` by the size of the data type. The division basically returns the number of elements represented before the passed `start`, which in practice are the number of validity bytes that appear before the passed `start`. The

subtraction is needed because the start index in the compressed data is the same as the one passed minus the number of bytes used to represent validity bytes.

For the **end** calculation is the same for all the types that have less than eight bytes. If the type has eight bytes, like **u_64**, then the offset that goes from **start** to **end** is doubled. The division discussed before returns the number of elements that are stored based on their size. For numbers that have less than thirty two bits, one **u32** is used, but for values represented with more than thirty two bits double the amount of **u32** are used, which requires the offset between **start** and **end** to be doubled.

The corresponding validity bit must be retrieved from its vector as well, in order to decompress the value. In this case, for values compressible in one **u32** then there is a direct mapping between the vector and the ordinal, so the first value's validity bit is stored in the zeroth position of the validity vector, the second in the first and so on. The value is then calculated using the simple division `start/data_type_size`.

For values that are compressed using two **u32**, the same idea is applied but the mapping is halved. This is because the space required to store the number is doubled, so the start is doubled compared to smaller types that use one **u32** only.

After calculating the respective start and end, the function needs to reassemble the original value. The value that is returned is an *owned* vector, that is instantiated using the size of the interval defined by the two indexes start and end. The vector is obviously a vector of **u_8**.

The function uses the iterator (Section 5.2) implemented for the structure, in particular this was the function that had the highest performance issues with the default implementation of skip. The function skips to **start** and iterates till **end**, by taking the compressed values.

The unsafe block is used to dereference a raw pointer to **self** in order to access the **validity_bytes** field during iteration. This was necessary because the Rust borrow checker could not statically verify that accessing **validity_bytes** (which is immutable during the loop) is safe while simultaneously holding a mutable reference for iteration. The raw pointer approach bypasses the borrow checker while maintaining memory safety, as we guarantee that **self** remains valid for the duration of the function and **validity_bytes** is not modified during iteration.

The function during each iteration extracts the bit from the validity byte corresponding to the value and pushes it to the vector instantiated as result. Then, for each value, the function groups bytes as **u32** and writes them to the end of the result vector. The validity bit writing happens once every two loops for types that are represented using two **u32**, in order to match their logical representation.

After reassembling the value inside the vector, the value is returned along

with two pointers, one to the validity byte and the other to the actual value start. These two pointers are used to implement prefetching on compressed indexes as well. Prefetching in this scenario loads compressed data that will be later required from queries. Pre loading compressed data can be more beneficial than loading uncompressed data, since one cache line can store more relevant information in the compressed version.

This is the most important function to access compressed data. It is clear that it brings some overhead that is composed by: the decompression time, that directly depends on the `bitpacking` library, which is pretty fast since it is highly optimized and makes use of `SIMD` operations, and the other main overhead is given by the reconstruction of the values and the access to the two vectors.

Splitting the two vectors, the one containing the data and the other containing validity bytes, is pretty reasonable since validity bytes can be compressed without any constraint brought by the data.

The downside of this approach is that data is not close to each other in memory and two allocations are needed instead of just one.

5.4 Motivation for bit packing

The bit packing choice comes from different factors. The first comes from constraints that were already discussed earlier, since the data structure is meant to be read heavy and that access time is a priority compared to memory usage, not every compression scheme works well with the given constraints.

For example common multipurpose compression schemes like LZ4, do not fit the requirements since they are slower and have much higher overhead compared to faster compression schemes like bit packing. Obviously the trade-off goes to main memory utilization that is acceptable in the use case. The other reason why bit packing was the first choice is its easy implementation and the existence of the `bitpacking` library that sets really high standards. It is indeed difficult to build an implementation that surpasses the library one, so implementing a wrapper that adapted the library to our use case was the best and most realistic option that guided us to the current implementation, making it as production ready as close as possible.

The idea behind the compression portion of the project is to allow multiple compressors to be interchangeable using a compressor trait in the index module. Using this approach allows to use different compression schemes based on their qualities. For example, since bit packing with the current implementation allows for high performance on random access, a random access trait is implemented, in this way it is possible to categorize different compressors as well.

In this way compressors are divided in categories based on the traits they implement, if a compressor fits fast complete vector decompression, then

it will implement a specific trait that will allow implementors to know its strengths and assume some performance guarantees.

After the bit packing implementation the development of other compressors from scratch started, like simple-9 and simple-16, but they are not complete yet and the focus moved to the index portion that needed much more attention.

In the benchmarking Section 7, lightweight compression will be analyzed in isolation, and the bit packing implementation developed in this work will be compared with alternative compression schemes. Their respective strengths, trade-offs, and suitability for different access patterns and data distributions will also be discussed.

This Section will describe how the hybrid implementation was developed, structural choices and changes that were made during the development.

Then the actual adaptation of the index and the overall components that compose it will be described along with the different difficulties and problems solved during the implementation.

```
1 {
2   "schema_name": <string>,
3   "schema_id": <string>,
4   "fields": [<FieldDefinition>]
5 }
```

```

5      {
6          "id": <integer>,
7          "name": <string>,
8          "type": <string>,
9          "storage_layout": <string>,
10         "unique": <boolean>,
11         "nullable": <boolean>,
12         "faceted": <boolean>,
13         "compression": <string>,
14         "items": <string>
15     }
16 ]
17 }

```

In this subsection the *index schema* construction, shape and implementation choices will be discussed in order to give a better understanding of how the overall index is structured.

The schema can be defined by hand, enumerating each field present in the index along with its properties. A field in the schema represents a column of the dataset; a set of all columns described in the schema represents a row of the dataset.

The schema is described using the JSON file format, which is pretty standard for such a use case. The other candidate formats were: TOML, XML, YAML.

The JSON format was preferred over the others for its high compatibility with a lot of different tools, which allows it to be more flexible for possible future integrations with other services and, on top of this, it uses a clearer representation compared to the other formats. For example, the TOML file format is better suited for configurations rather than the representation of big objects, XML looked too cumbersome for the use case and it is not as popular as it once was, YAML is not adopted as much JSON is and it is usually easier to have format issues when writing it even if it is easier to read from a human perspective.

6.1.1 Schema fields

The first field of the schema definition file is the `schema_name` field which is used to give a human readable name to the schema. The name field does not have any constraint and it has no use in the core logic of the implementation, it is just used to distinguish different dataset schemas without reading the *id* that can be hard to interpret for a human.

The second field encountered is the `schema_id` which is used to give a unique identifier to the schema in order to pair it with one or more datasets. This identifier is necessary because each dataset will have a schema, which is tied

to the structure of the dataset. One file can describe more datasets as long as they adhere to the schema's description inside of it.

The alternative implementation choice was to attach the schema as a *header* of the dataset in order to always have its description along with it. This second choice removes the possibility of sharing a schema between more datasets.

The `schema_field` is represented as a `string` type to give more room for `ids` choices. Using a `string` also allows for more readable *ids* if necessary.

The third field is an `array` of objects which are the core of the dataset description. The array collects all the fields that are present in the dataset. In short: a field, which is an element of the array, is a column of the described dataset.

Let's now describe how a single field is composed, fields will be also called columns sometimes to avoid confusion between JSON fields and dataset fields.

1. `id`, mandatory value used to give a unique identifier to each column. It is an integer and it is crucial for different operations applied during the execution.
2. `name`, value used to give each column a more human readable name. This field's purpose is mainly helping during the debug process and inspection of the code during execution, its value is not used for computations and is not meaningful for the implementation logic.
3. `type`, this value represents the type of the stored values. It is represented using a string that will contain the rust type that will be used for the representation in the data structure at runtime. If the schema is programmatically generated, then this field must be derived from the actual data that is inside the dataset. Types that are stored in the dataset must be convertible to the type expressed in this field.
4. `storage_layout`, this field represents how data is represented inside the data structure. The values can be either `columnar`, `row`, `row+columnar`. If the data is stored using the `row` layout then the field is stored in a row-wise manner, this means that all the row stored values in the dataset will be stored contiguously in memory. This type of layout is useful when multiple different fields need to be retrieved together, this type of access can leverage data space locality and have increased performance.

If the data is stored using the `column` layout then all the values of the field will be stored contiguously. This type of layout gives an advantage when a single field is accessed multiple times in a row. The advantage comes from the fact that all the values from the field are contiguous, giving again the advantage of space locality. This type of access, in the

row layout, has the disadvantage of having to access multiple “useless” fields that are contiguous to the memory that needs to be accessed, making the access slower in the specific scenario.

For the **row+columnar** layout, both the representations are used. The disadvantage in this scenario is the memory overhead of having both the representations, while maintaining the advantage of both the two representations mentioned before. Later the specific implementation of all the layouts will be explained in detail.

5. **unique**, this field is a boolean value that tells if the column has only distinct values (**true**). This information can be leveraged to optimize or guide the choice of the compression scheme.
6. **nullable**, this field is a boolean value that tells if the column contains at least one **null** value. This information can be used again for optimizations or choices about the compression scheme used.
7. **faceted**, this field tells if the column can be used for faceted search. Faceted search allows for searches that include ranges and different attributes based on the values inside a column. This type of search allows the user to narrow down the search to what exactly they are looking for. Having this flag set to **true** adds the column to the pool of possible faceting search values. Faceting search is usually empowered by the use of column store.
8. **compression**, this field tells the type of compression to use in the current column, for the current implementation only bit packing is available. The aim of this field is to put a specific type of compression method that fits better the given data. For example, bit packing usually works well with column stored values since they can be similar and so have less variation in the number of bits used to represent them, which is a weak point of bit packing compression.
9. **items**, this field is not present in every column of the dataset, in fact it is only present when the field is a **multi_value** type field. A *multivalue* field is a field that can hold multiple values for a single entry, in contrast to standard fields, which store only one value per row.

In this case the **items** field, contains the type of the items that are inside the multivalue type since the **type** field is already used to bring the information that the field is *multivalue* and cannot be further used to bring the information about the type of the values inside each row.

As it was already mentioned, the **JSON** configuration file can be either hand-written or generated programmatically.

During the development different *Python* scripts have been developed in

order to generate a suitable starting point for the correct JSON file. These scripts take a dataset as input data and write a suitable schema file for the given dataset. Some information can in fact be derived starting from the dataset and be used for the schema file generation.

Clearly not every information is derivable from the dataset, for this reason, interfaces are provided to safely modify values for each field, in order to always generate a proper JSON file that can be later used by the Rust implementation to create the data structure.

These scripts determine the appropriate types for the various columns. To achieve this, the first pass relies on the *pandas* library types, which serve as a good upper-bound approximation. This first pass is able to correctly identify, multi values, Booleans and strings that are already represented by the library types.

Integers though, are always represented as `i64`, which is clearly an upper bound that must be tightened to better fit the Rust type system. In order to do this, the script implements a function that analyzes all present values in each column and, based on the *minimum* and the *maximum* found for each column, it is able to identify the correct Rust type to represent each column. These types are then converted into strings in order to write them inside the JSON file.

As already mentioned the script is not able to infer the best compression scheme or if a column is better stored as `row` or as `columnar`, but some heuristics can be used to improve the script and increase automation for the JSON generation process.

6.2 Schema file parsing in Rust

The previous subsection outlined the creation of a formal JSON file used to represent the schema of a dataset ready to be stored in the Hybrid Index. This file though is generated starting from the dataset and just describes its properties, but, in order to be useful, it must be interpreted and used by Rust code.

In this subsection the mechanism used to bring those JSON information into Rust and the different decisions and techniques used will be discussed.

The module that is responsible for this task is the `schema_loader` module. This module's task is loading the JSON information in a Rust usable code, using different enums and structs that will represent all the information stored inside the different fields described in the file.

This information will be later used in the code to enforce properties of the dataset, like how different fields data is stored at the physical level, or how and if a field is compressed or not. This means that it is crucial that the information brought from the JSON is correct and that it is coherent with the dataset.

Any error in the `JSON` is reflected in the structs generated by this module and will propagate to other portions of the code. This module has no way to double check that information brought by the `JSON` are correct, which means that responsibility about the correctness of the fields is on the `JSON` writer that can be either manual or programmatic.

This basically means that the `schema_loader` module assumes the `JSON` to be correct and starts working from this assumption.

The first struct worth mentioning is `Schema`. This struct holds the schema name, the schema id and a list of fields that are the main structure of the index.

The schema name is used to give a more human friendly naming for the schema.

The schema id is instead used to uniquely identify a schema. The main purpose of the schema name field is making debugging easier, while the id field can be used in scenarios where multiple indexes are instantiated in the same execution to distinguish them and, for example, distribute queries on target indexes based on their ids.

The fields field is a vector of fields that is constructed based on the `JSON` fields and it is almost a one to one representation of those, but as Rust data.

This struct can be instantiated using the `from_file` method. As the method's name suggests, it allows to build a schema, starting from the `JSON` file.

After reading the file using the `serde_json` [30] framework, that was used on different sections of the code, the function makes use of the `RawSchema` struct.

The `RawSchema` struct contains the same fields of the standard `Schema` struct, with the difference that it stores a vector of `RawField` and not of standard `Field`. These raw structures are used as an intermediate representation between the parsed `JSON` information and the final structs.

The reason why these intermediate structs are needed is that the parsed `JSON` returns strings fields and not the specific enums that are then used to represent the possible value that each field can assume.

The main job of the raw structs is to convert these strings to the enums that will be later used by the final structs to construct the actual index.

To create an instance of `RawSchema`, the `from_str` method is used. This method deserializes the `JSON` into a `RawSchema` struct, ensuring through type checking that the data in the file conforms to the structure and constraints defined by the struct.

If the structure is not respected the function fails, not allowing to deserialize malformed inputs, this steps guarantee that the `JSON` adheres to the struct definition, but do not guarantee that its semantics stick with the dataset form.

This means that as long as the `JSON` defines types that can be deserialized, no errors occur, even if the dataset contains values whose actual semantics

differ from the specified types, as long as they can still be deserialized into another compatible type. Such situations can arise because raw data is involved.

As already mentioned, **RawSchema** contains a vector of **RawField** structs. The **RawField** struct, is identical to the standard **Field** struct, but it contains standard types instead of the custom enums that are needed to construct the index. The struct contains:

1. *id*: stores a numeric id used to identify and query the specific field in rows or columns.
2. *type*: used to represent the type of the fields, this is one of the field that differ from the standard **Field** struct because it is defined as a **String** instead of the actual custom enum.
3. *storage_layout*: another field that differs from the standard field, even in this case represented using a **String**. Represents how to store the field, either columnar or row.
4. *unique*: represented as a **bool**, it indicates whether all values in the field are distinct. This property can be exploited for optimizations; for instance, it can inform the choice of compression algorithm, as some algorithms perform poorly when the data lacks redundancy.
5. *faceted*: represented as a **bool**, determines if the field should be usable for faceted search.
6. *compression*: another field that differs from the standard field, represented using a **String** and determines which compression algorithm to use on the field.
7. *items*: this field is present only if the field is a multivalued field. If the field is multivalued, this field determines the type of the items that are stored by the multivalued.

These are the structs that are involved during the deserialization of the JSON file. Now the **from_file** function needs to pass from a **RawSchema** struct to a **Schema** struct.

To achieve this, the function iterates through each **RawField** and calls the associated function **for_raw**, defined for the **Field** struct, to generate the actual fields vector. This function calls the associated function for each field that was previously defined as a **String** to convert them to the correct enum.

Three enums are involved during this conversion:

1. **FType**, actual type of the field. Each possible value of the enum is a redefinition of a type in Rust. This was already described in Section 4.

2. **StorageLayout**, used to determine how the field should be stored. The possible variants are row, columnar or both.
3. **CompressionType**, used to determine the compression scheme. The possible variants are bit packing, no compression or automatic. The automatic version is not yet used, but the idea is to implement a feature where, based on the information given from the dataset, the best compression scheme is chosen automatically.

The conversion is performed by the implementation of the `from_str` method, which is defined for each enum. This method returns a different variant based on the provided string. By using this approach the function checks that passed strings are one of the variants, giving even more guarantees about the shape of the JSON.

All of the converted fields are gathered in a vector that is later used for the **Schema** instance that is returned by the method. Before returning though, another check is performed. Each field id is inserted inside an **HashSet**, in order to guarantee that ids are unique between fields. After this check the **Schema** instance is returned.

The **Schema** struct also implements some utility functions that are used to return the positions where fields are stored. There are three of these functions that are `get_columnars_only_positions`, `get_row_stored_positions` and `get_row_col_positions`.

Each of these functions returns a vector containing pairs of field ids and their relative position in the row. In later sections the use of these information will be covered in depth but the main use of these functions is to reconstruct rows, when an entire row is retrieved.

The last function that is worth mentioning in this module is the `to_row_descriptor_field` function which is used to convert a **Field** struct to a **DescriptorField** struct, implemented by the row version of the index. The initial struct is not enough because its main use is to pass from the JSON file to a Rust struct, and it is not possible to directly pass from the JSON file to the **DescriptorField** without this intermediate conversion. Later, during the index construction description this will be further discussed.

6.3 Index construction

This Section will describe how the gathered information from the JSON file are used in order to build an index coherent with the given specification.

The first implementation made was just for the row version of the schema, the hybrid version builds on top of this, supporting the definition using columns and not only a single row of fields.

This portion of the code is written in an executable file, so the code resides inside a main function, in order to test the instance of the index and run queries on it. Not every aspect of the file is described since it will be discussed again later and in more depth.

The first thing that the main function does is parsing the schema using the function described in the previous subsection, this leads us to a `Schema` struct that is passed to the `from_schema` function, which is the one that actually builds the schema and it is defined inside the `MultiDenseIndex` struct.

Inside this function several local variables and vectors are defined in order to allow for the construction. Two vectors are defined to store row and column stored fields separately, the other defined variables will be described during the function break down. Also a third vector is defined to hold which fields need to be compressed and with what compressor.

The function starts iterating through the defined fields in the schema. It is important to notice that this iteration assumes that fields are defined in the `JSON` file using the order that the user wants them to be stored physically. This means that the iteration order, also defines the order how they are stored in memory.

On each iteration, the function fills the different vectors defined before with the fields and makes use of another vector called `ordinal_mapping`. This new vector is used to keep track of the position of each field in the index. This is necessary since some queries may ask for a certain field based on their position as if they were all stored by row.

The ordinal row position is an intrinsic information for a default row store, because querying for the third column of the row is just a look up in the row, for the hybrid implementation that we made, this requires an extra data structure (a vector) to bring this information, given that fields do not preserve their definition order if column stores are used.

During iterations each field is matched based on its storage format type, row stored fill the row vector, column stored fill the column vector and row and column stored are inserted in both vectors.

Only for column stored vectors there is also a check for compression, if the field is set to be compressed, then its position in the column vector is stored in the `fields_to_compress` vector along with the compression method that should be used for it.

The filled vectors that come from the iteration are not yet usable for the index construction. The reason why they can not be used is that the index uses its own `Field` type which is different from the one that is defined in the schema. In order to make them usable for the constructions, two other iterations are made on both vectors to convert them to the index `Field` custom type. The two iterations are made using iterators and gathering the transformed values inside two other vectors.

As already described in previous sections, the hybrid implementation of the

index is based on the standard row. This basically means that the hybrid index is composed by multiple row indexes adapted to either represent by row or by column. In particular, row stored fields are all stored in a single row index that is in place stored in a vector of indexes inside the `MultiIndex` struct.

Column stored fields are instead represented as row stores that only have one field. Using one field only, practically makes them column stores, given how the row store index writes in memory, making all the data contiguous. Given this, the current construction of the index must act differently based on the store type of the field. Row stored must all be stored together as a single row descriptor for the first index of the `MultiDenseIndex`. In order to do this, one `DenseRowDescriptor` is constructed on top of the row fields that the function had just transformed.

From this row descriptor the function can just create the index by calling the appropriate `new` function, using the descriptor as a parameter.

For column stored fields, the function needs to create one single index for each field. To do this, another iteration is performed, where each field is converted to a row descriptor and an index is generated from that. These indexes are then gathered into one single vector, where the first one is the row stored index, followed by the column ones which are ordered by the definition order defined in the `JSON` file.

This function uses several iterations that could be combined together to increase efficiency, but, since the index should not be constructed often and the number of fields does not scale much, readability was preferred over performance in this case, since the performance gains from combining and reducing vectors would be minimal and just result in less readable code.

The function returns an instance of the index, filled with the variables that were defined locally, their use will be explained in later sections.

6.4 Hybrid index structure

Before explaining how the index is populated, it is necessary to discuss the details of its structure in order to better understand the design choices made for the writing and reading implementations.

The struct that implements the hybrid approach is `MultiDenseIndex`. The struct contains several fields, all of which enhance the capabilities of the standard row implementation upon which this struct is built.

Every field will be described below, justifying each of their use in the overall context.

1. `indexes` is a vector of type `IndexType`, and it constitutes the core of both the struct and the hybrid approach. This vector stores all the indexes used for fields stored in both row and column formats. During development, various strategies were explored to unify the storage of

all indexes. These included the use of generics with monomorphization as well as dynamic dispatch. Ultimately, the final solution adopted the use of an `enum` to encapsulate all index types.

The first idea was to implement a trait with the core functionalities of all types of index and then build a vector of generics implementing that trait. This solution was reasonable, but the issue came from the fact that compressed indexes and normal indexes did not share the same core functionalities and the implementation of a trait, with methods that covered both core functionalities would have made little sense.

The problem for the implementation using monomorphization was that it required all elements inside the vector being of the same type, which was not feasible since the vector was designed to store both non-compressed and compressed indexes, which are two distinct types.

The natural solution to this problem was dynamic dispatch, which allows multiple types to coexist within the vector. However, it introduces the limitation that only methods defined by the specified trait can be invoked on the objects. This could be addressed by implementing all the functions required by both types in the trait. This approach was discarded because it would require implementing empty decompression methods for the non-compressed index, which we considered poor practice.

The current solution makes use of enums, which can be checked at compile. The enum has two variants: `Normal` and `Compressed`. Both bring an index of the corresponding type.

The main downside of this approach lies in maintainability of the code base. Given that the variants are not expected to grow significantly, the solution appears reasonable.

2. `field_to_index` is an `HashMap`, that if queried with a column stored field id returns the position in the vector of indexes. This map is useful to retrieve the exact index for a certain column.
3. `columnar_only_positions` is a vector that contains the pair of field id and the position of that field in the row. Useful during reconstruction of rows and other similar access patterns.
4. `row_col_positions` is a vector that contains the positions in the row of `RowCol` stored values.
5. `ordinal_mapping` is a vector containing the field id and its relative position within the index. As columnar storage can only have a single column, the pairs will always consist of the id and zero.
6. `fields_to_compress` is a vector containing the field id and the type of compression needed for that field. Only fields that need compression

are stored here, if a field does not need to be compressed it will not be stored.

7. `row_positions` is a vector that keeps positions relative to the full row of the complete schema of row stored fields.
8. `fields_num` contains the number of fields that are in the overall index.
9. `row_mask` is a temporary vector that is used during row reconstruction. It is flagged using the `serde skip` tag to avoid serializing it. The vector is not serialized because its main use is to avoid re allocations that otherwise are needed for every row reconstruction, which is highly inefficient.

The struct also contains an explicit lifetime parameter, which is needed by the `GenericType` enum. This parameters comes from how the two variable types `String` and `Blob` are handled. Both these types are stored as vectors of `u8`, which basically are their bytes representation, but, since they can have different lengths that can span from small to large dimensions they must be handled with care.

The first row only implementation, had an issue about these types, it cloned the whole type when queried for them, which can be really inefficient when handling large strings or large blobs scenario that can happen frequently, especially with certain dataset that include descriptions or similar types of information.

The current implementation avoids cloning these types and instead handles them using references. The first attempt was to store a simple reference to the data. This approach though required to store a reference to the bytes array inside the `GenericType` for both the strings and blobs.

The main issue faced though was that the current write interface takes a reference to a `GenericType`, which stores an owned value and copies it inside the row that will be later added to the index. If a reference is passed, instead of the owned value, the interface should be changed and another type to write with owned data should be added.

Since owned data is needed to write and borrowing is needed for reading, instead of changing the interface, the smart pointer `Cow` was used instead.

This smart pointer allows to either borrow the data or to own it, which is exactly the behavior needed for this scenario. Because it introduces no overhead and adds no extra complexity to the code base compared to the reference version, `Cow` was used in this implementation.

The main advantage earned from this strategy is to avoid changing the already tested interfaces that would require refactoring on other portions of the code, like the data generation functions one.

The data generation simply creates random vectors of `u8` using the random library and instantiates the enum variant for blobs and strings with the

generated value. If normal references were used in that context, the function would not have worked, since the random value is dropped immediately after the function exits and passing a reference to it would result in a dangling reference which leads to not even compile. By using `Cow` instead, the value is passed as owned and later, when reading, a reference can be returned using the `borrowed` associated function of `Cow`.

To briefly summarize `MultiDenseIndex` is a struct that wraps a set of indexes that are used to represent the data using a specific format described in the schema description. This struct is designed to store the various row and columnar indexes, delegate the query to the specific index containing the requested data, and then return the computed result.

Interactions with this index can be different from writing to reading. Based on the interaction different approaches are made. In the following sections we will discuss how to interact with the index both for writing and reading information. The following Section will describe how to populate an empty instance of the index.

6.5 Populating the index

The function described in the previous Section, returns an empty instance of the index that now needs to be populated with actual data.

The first implementation that was used for testing, populated the index using generated synthetic data. In order to generate this data the `GenericType`, which is the custom data type defined to bring an actual value, implements a function called `random_value`. This function generates random data based on the type that calls it. For variable length fields like strings and blobs, the generation was made using a file to define their lengths and then random values, for each byte, were generated.

This data generation was used during early phases of development to test the index behavior.

Synthetic data is useful in development scenarios because it allows developers to test specific features and capabilities of the data structure. Synthetic data also allows to perform stress tests and other checks that are needed during development.

This was the case also for the project, data generation was used to do multiple tests during development and also allowed to spot different bugs present in the original implementation. For example, to test bit packing compression ratio, only small numbers were generated to test if bit packing was compressing as expected. This approach allowed to find the issues already described in the previous Section about compression 5 and also allowed to spot an issue about the row index implementation, which failed when only one boolean field was used as schema.

Generated data is not enough though since it often fails to capture patterns

that are present in real world datasets, which are crucial for evaluating the true performance characteristics of the data structure in a production like environment, where the data set is built on top of real world data.

For these reasons the current implementation uses real world data that was used to benchmark and test the data structure. The current implementation loads datasets using the `csv` file format, which is ideal to represent datasets defined by rows.

A `reader` is instantiated to read the dataset row by row. Each column value in the row is then read by the function that needs to convert it in order to generate a row with proper types that can be added to the dataset. To convert each value a function is defined inside the `GenericType` struct, which is the type used to store actual values for custom types. The function takes a schema field type and a value and generates an `FType` that corresponds to the schema type passed. The `FType` will contain the passed value.

All the converted values are gathered together into a vector and, if data is in the correct form and nothing fails, it is considered as a row ready to be inserted in the index.

Inserting a row is not trivial because each field must be inserted in the corresponding index. For example, if a field is stored as row, then it must be part of a row that contains all the row stored fields that is then inserted in the first stored index inside `MultiDenseIndex`. If a field is stored as a column it is a complete row of a single index that stores the column and the insertion should be redirected to that particular index.

The `add_row` function coming from the `MultiDenseIndex` struct is the one that takes care of this logic. This function takes as a parameter the row of values that has to be inserted in the index and then applies the aforementioned logic. The first thing is the creation of a temporary vector that will be used to insert the row stored values inside the first index.

To fill this vector, the function uses one of the attributes of the struct, which is `row_positions` vector, that determines which positions of the row should be stored as a row. By the use of an iterator along with a `map`, values from the passed row are cloned inside the temporary row vector. Using `clone` allows to take a reference of the row to insert, avoiding to take ownership.

After gathering all the row stored fields, the row is ready to be inserted. To insert the row a simple check is made: if the type of the index is `Compressed` or `None` the function simply panics. The current implementation does not allow to compress row stored indexes yet. If the index is of type `Normal` then the row is directly inserted.

This insertion covers only the fields that are row stored. If some of the fields are defined as columnar, then additional logic is executed.

Before describing how column store fields are stored, it is worth mentioning that insertion can be implemented with a more efficient approach which involves specifying directly row stored fields and column stored fields. This

type of approach would reduce by a lot the time spent in this function, since logic for the lookup would be eliminated and the function would reduce to a simple insertion.

The current implementation though takes directly a full row from the dataset. This is a better interface obviously, that makes insertion slower, but easier to read and more reasonable in terms of code shape.

Having concluded this brief digression, we now return to the discussion on the insertion of columnar stored fields. Fields and positions for each column field are stored inside the `columnars_only_positions` that allows to extract each needed field from the row to insert to add it to the specific index. With this approach the function redirects each single field to the specific index instantiated to store that specific field.

During the iteration there is also a check for the type of the index. If the index is a compressed index, the function panics because compressed indexes are assumed to be immutable. The question that could arise from this statement is, how to populate a compressed index then. Given the non mutability assumption, compressed indexes are built from normal indexes. All indexes are treated as normal index, populated and then compressed in one way, using the dedicated field in the structure to find out which indexes should be compressed and how.

The overhead of columnar fields is clear, for each field an access to the vector is required in order to redirect the insertion to the correct index. Clearly, during each iteration, the single value is inserted in the index using the same insertion function used by the row index.

After all the insertions for each field of the row are done, the `len` field is incremented by one, since one full row from the dataset was inserted.

The `add_row` function is called for each row coming from the dataset and, when it is done inserting one row, a check is executed for each insertion. This check compares the last row in the index, calling the `get_row` function, and the row coming directly from the dataset.

This check can be toggled on or off based on a passed parameter to the command. This is not the only check that is executed though, right after inserting all the rows from the dataset, then the compression step happens. Each row and the corresponding inserted line in the index are saved in an extra vector. This vector is later used to check that also the compressed version of the index answers with the correct rows.

In the next Section the compression handling will be discussed. The current result of this populating procedure is the index, with both rows and columns, but without compression yet.

6.6 Index compression

After all the rows have been inserted, the index can be compressed. As already explained before, it is assumed that compressed indexes are not mutable, so it is more convenient to populate the indexes and then compress them in one go, rather than inserting in the compressed versions that would require bigger overhead in insertion (with the current bit packing compression, the overhead is purely based on the compression algorithm adopted). The `MultiDenseIndex` struct implements the `compress` function that takes care of this procedure.

The function knows which indexes to compress using the information stored inside the `fields_to_compress` vector field. Two extra variables are allocated inside the function to keep track of the compression gains coming from the procedure. One of the variable is used to keep track of the dimension before compressing and the other keeps track of the size post compression. These two variables are used to calculate the compression ratio after applying the compression.

During the iteration a `match` statement is used to check what algorithm should be used to compress the given index, this guarantees that only allowed compression algorithms are used and this is where extra code for next compression schemes should be added.

After matching the compression algorithm the function performs an additional check to determine whether the index contains only one column, since compression is only allowed on column stored field for now. If the compressed field is stored by row the function panics.

Now the function reaches the core of the compression, here some design choices about the code structure will be explained.

As already said there are two different types of indexes: `DenseIndex` which is the row storing index without compression and its counterpart that uses compression. The latter is constructed using the `from_compressor` function. This function builds the index starting from a normal index and a `CompressorWrapper`.

We now describe the design choices around these for the compressors which may not look intuitive at first.

The idea is to allow this compress function to operate with whatever compressor is passed, in order to be as flexible and scalable as possible in terms of compressors implemented. To do this the `CompressorWrapper` enum is defined. This enum is designed to contain, in each of its variant, a different compressor. In the current implementation, for example, the only variant available is `BitPacking(BufferedBitPacking)`, which is the variant for the bit packing compression, containing the struct that exposes the methods for compression.

The enum implements the `DenseIndexCompressor` trait, that requires to implement the necessary methods for the decompression that are implemented

per compressor. The functions required to be implemented by the trait are:

1. `get_decompressed_rows`
2. `get_rows_number`
3. `decompress_range`
4. `get_compressed_rows_ref`

Each of these functions contain a match statement, that matches `self`, this means that based on what variant of the enum is used the function matches a different arm.

With this approach creating an instance of the enum with a certain compressor will keep the same interface provided by the trait, but will change the behavior of the function, based on the compressor choice.

The actual implementation calls the functions from the instance of the struct passed to the enum inside their matched arm. This allows for a total control of the interfaces of the passed instance but still gives a unified interface for all the compressors, without limiting the underlying implementation.

To make things clearer let's go through the current implementation for bit packing.

In order to generate a compressed index a `CompressorWrapper` instance is created using a `BufferedBitPacking` instance. This is generated conditionally based on the matched compressed type required by the schema. So here, the actual compression is handled by the `BufferedBitPacking` instance, while the `CompressorWrapper` allows to have a stable and common interface among the different possible compressor. The interface is defined by the trait and, whenever a method of the trait is called, the `BitPacking` variant is identified by the match and the specific decompression logic for the bit packing decompression is called using the instance. This does not force to implement all compressors with the same interfaces, since some might need extra steps or a different logic.

This approach gives more flexibility than a simple trait implementation on compressors, but has the draw back of higher maintenance costs in terms of code base, since two match in two different files have to be handled during the implementation of new compressors. This drawback is minimized by the fact that the number of compressors to be implemented does not scale much and makes then the choice reasonable.

The specific implementation of the trait for the bit packing is not covered in details since it reduces to calling functions that were already explained during the bit packing Section 5.

The compressed index differs from normal indexes, not only on how it stores the data, but also on how it is structured. The index does not store rows directly in its fields, but it stores instead the compressor described above. The

`DenseIndexCompressor` trait, in fact, requires to implement all the functions needed to allow the compressed index to interact like normal indexes do. For example, in order to get the space consumption of the data structure, the size comes from a function call made on the compressor that returns the number of rows that are stored in its compressed buffer. The same happens for decompression functions, that are redirected to the underlying compressor. These methods should never change, the changes should be made at the compressor level, in order to reduce the code changes made only on new files implemented.

The `access` and `get_row` methods change from the standard index only for the fact that this implementation calls the range decompression method from the compressor instance, so that the decompressed bytes are returned based on the passed indexes. The current implementation was described in details in Section 5.

Another change was made to the prefetching logic, since data is not laid out in the same way as in standard indexes. There are three methods that are implemented to use prefetching:

1. `prefetch_validity_map`
2. `prefetch_lines`
3. `prefetch_fields`

Names are pretty explicative, each of method allows prefetching for a different type of data of the different rows.

The first two make use of a function implemented in the `utils` module called `prefetch_read_T0`. This function uses a generic to take a reference to some data that can be loaded in cache and an `usize` offset used as a second parameter. The offset is needed to indicate which data should be loaded in cache from the data passed.

The function uses a raw pointer that initially points to the start of the passed data, it is then moved to point to the passed offset using pointer arithmetic.

The function has also an architecture specific behavior, for example it is compiled only if the prefetch feature is enabled and the target architecture is `x86` or `x86_64`. The prefetching intrinsics vary depending on the architecture, but they all serve the same purpose: hinting to the processor that the data pointed to will be used soon and should therefore be loaded into cache.

This function code was not implemented by me but was already present in the base row only implementation, but it was important to describe it since it is used by the adapted prefetching functions that are used by my implementation.

The first function was adapted by passing the data coming from the compressed index. In the standard implementation data is stored using one

single array, but in the compressed version the validity bytes are instead stored elsewhere in another vector that needs to be loaded separately from the data.

Same idea for the lines prefetching, in this case the data is stored in one place, but the compressed version of the lines are loaded. This leads to loading more meaningful bytes that can be exploited by the CPU compared to the non compressed version where less meaningful data is stored in the cache. In this case lines are passed as a vector of `usize` and each of them is loaded separately using their values as an offset.

The last function loads using a different function that is `prefetch_read_NTA`. This function is really similar to the already described `prefetch_read_T0`, but in this case the `NTA` intrinsics are used instead.

There is a difference between these two type of operations, `T0` fetches into all levels of the cache memory, while `NTA` fetches using the non temporal hint. The non temporal hint informs the CPU that the data will not be reused soon. This avoids pollution of higher level caches and limits the storage to lower levels of the hierarchy and allows for faster eviction of the data that is known to not be useful in the near future.

The standard function here loads the fields along with their validity bytes, which are stored in the same vector. The compressed one loads from the validity bytes vector and from the compressed data vector, like in the other scenarios.

To sum up, this Section described how the index handles compression and the generation of the compressed index. It was described now, since it is the last step that is executed in order to build the overall index. After the compression of the various inner indexes is done, the build function prints some information about the resulting index and the time used to construct it, along with its length and space usage. The final operation that is executed is serialization.

It was not made explicit during the description of the various components of the index, but every struct that is relevant to the index derives the `Serialize` and `Deserialize` traits. This highlights another advantage of compression that is not only reducing the space in main memory, but also on disk memory, which even if highly scalable can still be a factor in certain scenarios.

This sections concludes the construction of the index that now is serialized to disk using `bincode`. The following sections will describe interactions with the index and how it actually retrieves data during execution.

6.7 Reading from the Hybrid Index

After the index is constructed and populated with real data, it can be queried. The read functions implementation is based on the standard index implementation that was already described in Section 4.2, but it differs

on how data is then reconstructed.

The main difference comes from the fact that the standard implementation has to deal with only one index and only one vector of data. The hybrid index contains a set of indexes which makes things a bit different in terms of the logic.

The implementation kept all interfaces with the same name in order to make integration with already existing components easier. The two functions that allow the access to the index are:

1. `access`
2. `get_row`

The first function, `access`, takes a `row_id` and a `field_id` as parameters and it is used to query the index for a specific field of a specific row. The standard implementation just queries for the field using logic that was already explained in depth before.

The same could be done here, but only when the function discovers in which index the field is stored. To do this some information stored in `CompressedDenseIndex` is used. By querying `ordinal_mapping` the index id and the relative position inside the index are retrieved and the index can be queried. An additional step is required, which is matching the index type, since the retrieved index can either be a compressed or a normal one. This pattern choice is forced by the decision of using an enum over generics, which forces the pattern match that is though more efficient compared to dynamic dispatch at runtime.

After the match is done the relative access method is called and its values are returned as answer to the query.

The other function `get_row` is quite more complex, because it requires to build the full row from the different fields. This function is not required by the implementation, but can be useful in different scenarios and it is even used, as it was already showed, for checking that the inserted row is correct when queried back in the build process.

A work row is instantiated at the start of the function, this is used to reconstruct the overall row, that is later returned. A pointer to the indexes vector is generated in order to iter trough them during reconstruction. Since the returned values hold references to strings and blobs, rather than the whole objects, the borrow checker has some issues proving mutable references while iterating through indexes, as it will be later explained, and though an `unsafe` block wrapping the code is required. Some refactoring could remove the solution, that was used to make the code more readable.

The unsafe portion of the code starts with a check that panics if the number of indexes inside the data structure is equal to zero, which means that the index is malformed and has no indexes inside of it, which means that it can

not be queried.

After the check is passed, another vector is instantiated to collect the values that are stored inside the first index, which in turn contains row stored fields. After this new vector is filled, an iteration starts through it in order to put the row stored fields in their correct position in the overall row. To do this the `row_positions` field is used, accessing it using the iteration index, operation which returns the correct position in the overall row. The correct position index is used to fill the `work_row` with the row values.

After this iteration, row values are stored in the `work_row` in their correct positions and only column stored fields are missing to complete the row.

To complete the row, another iteration starts through the field `columnars_only_positions`. This iteration gives two values, the `field_id` and the position in the resulting row.

To find the correct index, the `field_to_index` is queried, using the `field_id`. Another check is done to this returned value, if the value is zero, then there must be an error, since the index at position zero stores only row values, if the index goes out of bounds then there must be an error as well and the execution panics if either one of these conditions is met.

After passing this check then the correct index is retrieved and, based on its type, the `get_row` function is called to get the columnar value.

Finally the retrieved value is stored in the position given by the iteration variable inside the `work_row`. When this iteration is done as well, the `work_row` is returned as an owned value, in order to match the same implementation made in the standard row based approach.

The `unsafe` block here is required by the two iterations. The issue lies in the fact that the borrow checker cannot prove that the code is accessing always different indexes in the vector, so it considers it as a borrowing of the whole vector which ends up being considered borrowed twice in this scenario, thing that is not allowed in Rust. The raw pointer does not participate to borrowing rules, so works immediately compared to other safe solutions that could be applied.

6.8 Querying the index

This Section will describe how the index is actually queried, basically putting the just described functions to work in order to get results and benchmark the capabilities of the data structure.

The querying of the index happens inside the `perf_multi_index` file, which is a binary file that handles the benchmarking of the structure.

Inside this file, queries are executed in order to extract the required data. Queries come from a set of queries that are always used, to ensure consistency between different datasets and benchmarks. Later in the Section the code used to adapt the queries to different datasets will be described in depth.

The file will not be completely covered, only the query portion will be explained in this Section, since it is relevant to how the index is accessed. The whole benchmarking system will be described later in its dedicated Section 7.

After the index is built, it is ready to be queried. As it was already described, the queries are located inside a serialized and dedicated file, that is parsed using the `parse_queries` function.

The deserialization of the queries file returns a vector containing the custom type: `QueryLog`. This type is an enum that describes the semantics of different queries, that will now be described. Note that not every variant is meaningful in the current implementation, but just a sub set of them will be relevant to the execution.

The queries variants are:

1. `ReadField(AccessType)`, comes with another custom type that defines the type of access that should be used in when reading the field. This variant is basically used to access one field in the dataset row. The `AccessType` variable that is brought with the type describes how and what field should be accessed in the row. `AcessType` contains two variants that are: `Dense` and `Sparse`. Both variants bring an `usize`, which represents the id of the field required to query. Later in the Section the difference between dense and sparse will be described, as a preliminary overview they differ on how they access the validity bytes.
2. `NextQuery`, not yet implemented and defaults on a shared case.
3. `NextDocument`, this variant is used to conclude the querying of the current document. Each document, which is a row in the dataset, is queried for one or multiple fields each time. The set of queries only goes forward in querying the documents. This means that if a document is queried for multiple fields, then the queries are all sequential, having a good locality, which is a good property for accessing. After all the fields are retrieved this variant allows to close the “transaction” for the current document and go for the next one that is passed as a `usize` to the variant. Multiple of these variants can happen to simulate the skipping ahead between documents.
4. `GenerateResponse`, not implemented yet, but should be used to return information about the previous querying.
5. `DocumentWasSelected`, not implemented yet, defaults on the shared case.

Before queries are executed, an iteration across the query vector is executed, where each query is matched with their variant.

The first match is for the `ReadField` variant, which is skipped if the given

document id is greater than the index length. If the document id is valid then there is an extra match that is done for the type of access.

The main difference stands in the fact that for dense accessing there is a prefetching for validity bytes as well, while for sparse accessing there is not. This is because, for dense stored values, all entries are there. This means that null values are present and so, using the validity byte can reduce useless accesses to these null values that can be avoided without even looking up in the row.

During this iteration more information are gathered, for statistics and benchmarking, that will be later explained in depth.

The second match that is done for the query type is for the **NextDocument** variant, which holds a document id as well. This match concludes the fields fetching for the current document. This case indeed pushes a field vector to another, to conclude the current “transaction” and clears the field vector right after. Later this procedure will be explained in detail, but for now the key point is that this type of query type concludes the current document transaction and sets the new current document to be queried to the one that is carried inside the variant.

The last case is the default one that is used for all non implemented variants. This case just logs the existence of the queries and clears the fields vector, basically closing a transaction like the next document does. The difference though is that no new document is specified, so a **NextDocument** variant is needed to keep querying the index.

The result of this operation is the **accesses** vector filled with queries that can access the index. This vector is then scanned and, for each query, the **access** function is called to get the values.

The rest of the file regards benchmarking cache hits analysis.

6.9 Queries adaptation

In order to have uniform and standard benchmarking across different datasets, the set of queries used to test the index was always the same. This set of queries were generated before using a specific structure in order to satisfy certain properties. The first property was already discussed and requires the query to always searching ahead of the current index. The other structural property is closing the transaction for a document before going to the next one. If this property is not satisfied then a document would be queried twice for the same field.

The query set also has some skips between documents, this means that there can be multiple **NextDocument** queries without any field query between them. Another motivation, beyond avoiding the redundancy of rewriting queries for each dataset, was the risk that creating a separate set of queries could introduce bias if the generation process were imperfect, potentially leading to subtle issues during benchmarking.

This serialized queries were given by Ebay and they are based on their datasets, which had several columns in the order of around one hundred fifty fields. In the initial phases of the development, the assumed arbitrary number of fields given by the queries files was not a problem, it just required to create a schema file descriptor that matched the number of fields required. When real datasets came into play, the number of fields became an issue. Finding real data sets big enough, in terms of rows was not an issue. Finding real data with the amount of fields required by the queries file was not as easy.

This problem lead to the solution of adapting the queries to the real datasets found. On top of the afore mentioned problem datasets are required to have a good amount of numerical fields storable using the column storage layout and that can lead to a good compression ratio for bit packing, in order to benchmark the new implemented features.

To adapt the queries the `adapt_queries` function was implemented for the `QueryLog` type.

The function takes the query vector and the number of fields of the target dataset. This means that the number of fields passed is the smaller between the two, in order to adapt the queries to smaller dataset. The function assumes that the used dataset has less columns than the original one, this assumption comes from the lack of datasets with enough fields rather than the opposite scenario, which leads to not ever querying a certain set of fields. The second problem was not faced during development, so it was not addressed and is not in the scope of the project.

The function initializes an `HashMap` that is used to map fields that do not fit in the smaller dataset, with the ones of the dataset. This basically means that fields of the smaller dataset will be mapped with their id plus a set of other fields that are not in the dataset.

In order to make these changes the function goes through each query and, whenever it finds a field that is not in the smaller dataset, it matches it with one field of the smaller one, by adding a relation in the hash map.

The field coming from the smaller dataset is chosen by a simple modulo operation. The modulo operation grants that the distribution of accesses remains the same.

For example, imagine a set of queries where ninety percent of accesses are made on one field that is not in the range of the smaller dataset. Using an approach where queries that do not fit the smaller dataset are discarded, clearly changes too much the distribution of the queries set.

Another possible approach that was considered, was to evenly distribute out of range queries between fields of the smaller dataset. This approach though, would clearly change how the index is accessed. For example, this method would completely remove the stress accessing on the single field,

which can instead exploit cache and lead to valuable considerations during performance analysis.

The modulo distribution keeps this ratio of accesses between the fields and allows to resemble the original query set, but in a smaller dataset.

The function does not return a new query vector, but modifies the passed one in place using a mutable reference to it.

The only variant that matters in this function is `ReadField`, there is no difference between the dense and the sparse accessing.

This adaption function is called only whenever a dataset not compliant with Ebay’s queries is used.

7 Benchmarking

This Section describes the benchmarking process applied to evaluate the results of the various implementations developed in this work.

The benchmarks aim to demonstrate the advantages of blocked compression schemes, highlighting how bit-packing and block-based techniques outperform alternative approaches for random access across the different required access patterns.

The benchmarks will also examine the index under both general queries, accessing row stores and columnar data, and queries on column-only datasets. The latter specifically illustrate the benefits of columnar storage and the performance gains achieved through bit-packing compression.

Standard profiling and benchmarking procedures were applied and described in this Section, as they were essential to identify subtle implementation issues that would otherwise have gone unnoticed. The tools used include both internally developed utilities and well-known external libraries in the field.

The primary metrics of interest in this Section are access time, cache hits, and memory consumption. The analysis begins with a detailed examination of the compression implementation as a standalone component and subsequently demonstrates its impact within the full index, serving as a practical use case to validate the effectiveness of the approach.

7.1 Benchmarking process and used tools

All the benchmarks were executed on a remote machine hosted by the University of Pisa.

System specifications:

1. CPU: Intel(R) Core(TM) i9-9900KF CPU @ 3.60GHz
2. Cache values:
 - (a) L1d: 256 KiB (8 instances)
 - (b) L1i: 256 KiB (8 instances)

(c) L2: 2 MiB (8 instances)

(d) L3: 16 MiB (1 instances)

3. Tests are executed on one core using:

(a) `taskset -c 0\allowbreak\ cargo run -release\allowbreak\
-bin perf\allowbreak\ bit packing`

7.1.1 Index benchmarking procedure and tools

The benchmarking logic is written into executable files. There are two major files used for benchmarking because the hybrid index needs some special handling. The core logic and the applied strategies remain the same, so the hybrid version will be described in this Section.

The logic is inside the same file discussed in the queries Section 6.8. As described in that Section, the first thing that the file does, during execution, is creating an index instance and then it deserializes and adapts queries for the dataset.

This version of the file is meant to benchmark the data structure using a real dataset.

The difference with the already described approach, used for synthetic data, is that there is an additional step used to adapt the queries vector based on the dataset size. This step is ran once and the resulting adapted queries vector is then serialized.

The choice of serializing came out since both the adaptation process and the overall benchmark can take a while, given the usual high number of queries executed. Serialization allows to just read once per benchmark, rather than adapt the original query vector for each run. Obviously if the dataset changes, the adapted query file must be regenerated and serialized again.

In the previous Section the generation of the **accesses** vector was discussed. In the same iteration where this vector is filled, several other metrics about queries are collected.

The most basic metrics that are tracked are the number of dense accesses and the number of sparse accesses. Both of these metrics just have a simple variable that is incremented whenever a query of their type is matched.

In the same match case the doc id length is increased. This variable is used to keep track of how many fields are accessed per document on average. In the current implementation, this value is only increased for the dense case. When sparse accesses will be implemented, the increment will happen on their branch as well.

There are also two extra simple metrics for both dense and sparse accesses that take care of how many fixed and how many variable fields are accessed. Variable fields are **blobs** and **strings**, while all the other are fixed length fields.

Then in the dense case matching, the `fields` vector is filled. This vector keeps track of the fields that are accessed per document, it is indeed emptied whenever a `NextDocument` or any other query not aiming for access is called. The last important value tracked inside the first iteration is the `lines` vector. This vector simulates the cache lines that need to be stored for prefetching, this mechanism will be described in details later.

The match case for all the queries that are not meant to access the index, like `NextDocument`, contains the logic to fill the `accesses` vector. This vector stores all fields accessed by queries. Fields accessed are stored inside the `fields` vector, as stated earlier, and inserted using the `add_slice` method, right after the `fields` vector is emptied.

Another use of the `fields` vector is to keep track of the frequencies of accesses per number of fields, to keep track of this other metric an `HashMap` is instantiated to store the fields lengths as keys and frequency of appearance as values.

In this match case a similar operation is applied to the `lines` vector, which is added to the `cachelines` vector and then it is emptied whenever the iteration is done with the current document.

All of these operations are guarded by a check that guarantees that queries are inside the index lengths bounds, in order to avoid queries to go out of the allowed length or similar issues.

To recap, after this preliminary iteration is done, the gathered metrics that are presented to the user are:

1. Time spent to gather this set of information
2. Number of documents accessed
3. Number of queries that will be executed
4. Number of documents that will be accessed
5. Number of dense and sparse accesses in total
6. Number of dense and sparse accesses for variable and for fixed lengths

After this first iteration, used to keep track of the basic metrics described, the file proceeds to execute the queries along with some important profiling information.

The file keeps track of cache related information using both hardware and software monitoring. Hardware monitoring and profiling is quite standard and will be described later in details.

Software monitoring is another well known approach that deserves a little introduction before presenting the specific implementation used for our use case.

As stated in [7], a software cache simulator is a piece of software dedicated to simulating a particular cache architecture for different purposes. The main use is to test new cache policies, without the need of building physical prototypes, in order to avoid extra cost and high times of construction, that can clearly be high and complex if applied during the research process, where testing is crucial and unavoidable.

The other main use case of a software cache simulator is testing the performance of a given program executed on a given hardware. This allows to test the given program without the necessity of owning different architectures that might perform different for the specific program.

On top of this, sometimes, hardware monitoring is not enough, since it might not be present in certain scenarios or might not give the specific metric required. A well structured and precise software cache simulator can give insights about the behavior of the cache for the specific data of the program. In the current implementation a cache simulator that can be further tweaked for more insights is present. Most of the information come from hardware monitor but it is still worth mentioning the software implementation as well. To implement such piece of software, a struct is defined to keep track of different meaningful values. The first field is an `HashSet` used to keep track of cache lines. The second field contains the current document that is being queried and the other two fields are used to keep track of hits and misses.

Two instances of this struct are created, one for variable fields and the other for fixed fields.

The crucial function for this struct is `address` that basically handles the computation and storage of cache lines.

The first value that is computed is the cache line for the value. The address is just the offset in bytes divided by sixty four, which is the standard length of a cache line in modern processors.

After computing the address, a check about the current document is executed. If the current document is not the same, cache lines are cleared, since the execution moved to a new document.

Finally the function checks if the corresponding cache line is in the `HashSet`, if it is a hit then it is reported, otherwise a miss is reported and the line is loaded.

This struct simulates a per document cache system. The results and benchmarks presented later in this Section use the hardware level reporting system since it has all the information needed, but it is still possible to enable this software implementation by passing the `stats` flag to the command, which is otherwise not executed.

Now let's describe how hardware profiling is executed. After the creation of the two software caching instances, multiple caches structs for hardware tracking are instantiated.

The hardware counters used are:

1. `L1D_ACCESS`, L1 cache read accesses.
2. `L1D_MISS`, L1 cache read misses.
3. LL cache hit on the last level cache, which is usually level three.
4. LL cache miss on the last level cache.

These are fairly standard metrics that are used to evaluate the cache performance for a given program. All of these structs are used inside a `group` that is used to aggregate these metrics in order to access them easier.

This `group` comes from `perf_event`, which is a Rust crate that allows for easy use of the `perf` command inside Rust. `perf` is a Linux command that is used for kernel level profiling, which enables fundamental profiling metrics that are required to profile a given program.

The last hardware monitored information is about CPU cycles, which is added to the same group of cache information.

The last portion of the file, where the profiling happens, goes through another iteration. This iteration contains a fixed number of runs, which ensure that the reported values are averaged and not a single execution that can have strange values coming from several factors, which may lead to biased results. For each of these runs, both cache lines and fields to be accessed per document are loaded at each single iteration.

Cache lines are prefetched, using the already described `prefetch_addr`, while fields are queried using the `access` function. These access calls are wrapped in a `black_box` function that is used to avoid some compiler optimizations that could lead to incorrect benchmarking.

Before the set of runs start executing a timer is started. This timer is stopped right after all the runs are done processing.

The time reported by this timer is used to calculate the amount of time spent per access and the total time used to access the index.

All the tracked information mentioned before are reported in the last phase of the procedure.

This procedure was already present in the base implementation for the row based index and was just adjusted to work on the hybrid index, in order to compare and test its performance. This file is not the only tool that was used to test the implementation though. A `justfile` was also used to run different profiling executions easier.

A `justfile` is similar to a `make` file with the difference that it allows to execute arbitrary commands that can even not be related to building the project. The used `justfile` defines commands for building the index, executing the queries and its benchmark and a set of `perf` commands to specifically profile certain aspects of the project.

For example the file defines `perf` commands to profile branches, cache misses only, prefetching, stalls and similar.

Each of these recipes existed for the base implementation and, an adapted version for the hybrid index, was created for each command. The necessity of having multiple commands, and not a shared one, is given by the hard-coded files that need to be ran for the two different indexes. It made sense to keep them separated rather than join them in a single benchmark file, doing both.

Another relatively modern tool used to benchmark and profile the implementation was the flame graph [10]. A flame graph is a visual representation of how much time is spent on a single function call. It makes use of horizontal rectangles to represent the time spent on the function, which means that the wider the rectangle, the more time was spent during the execution of that function call.

This graph is built using multiple of these rectangles, where each rectangle is followed by another. Since each rectangle represents a function call, functions that are called from inside others add their execution time to the overall caller function execution time. The next level that comes after a rectangle shows how much time each external function call takes compared to its overall execution time, showing where execution time is spent.

This graph is useful since it allows to easily understand how much each call takes and allows to identify slower functions that need attention. As it was already discussed, some inefficiencies were found using this visual representation. Whenever a function call takes too much time, which is identified by being represented with a long rectangle, the visualization can be zoomed in, in order to see which external calls that are executed from that function are made, to find the potential bottleneck or implementation inefficiency.

For example, if function f takes too much time, it will be represented by a rectangle being larger than expected. To inspect what is taking so much time in the a function call, the next level can be inspected. Imagine now that b is called from a and it is taking an half of the overall a rectangle. If b is supposed to be fast, for example a fifth of the overall function call, the culprit of the slow execution call is either b itself, or another call made inside of b .

This kind of representation makes clear how time is spent during the execution, by visually highlighting bigger execution times with bigger shapes.

7.1.2 Compression benchmarks procedure and tools

Regarding the benchmarking procedure for the compression component of the project, it is comparatively simpler than for the index. This is due to the smaller structures involved, the higher modularity of the implementations, and the fewer interacting components.

All the benchmarks take care of compiler optimizations in order to give the most realistic possible results. Just like for the hybrid index, also for the compression benchmarks the black box hint was utilized.

Each benchmark that is presented regarding access time and similar information about time, is executed using the `TimingQueries` struct, which is a custom struct that is used to register the access times using the standard library.

This struct allows to register not only the various access times, but their minimum, average and maximum. For simplicity, only the average value is presented.

The average value is calculated on top of 100 000 queries, unless otherwise specified, for each benchmark ran on 10 different runs. For random access queries and similar queries that do not have a constant value, differently from the EBay ones used for the hybrid index, queries are regenerated each time, in order to give the most stable and most realistic result possible.

7.2 Compression benchmarks

As it was already stated, the main focus of the work, revolves around having efficient random access for integers sequences and the index implementation represents the real world use case for it.

The backbone work, to achieve such results, is the compression implementation. This Section will show how and why the specific compression that we used is able to achieve such results and why it is the best fit for the goal.

The compression algorithms will be tested on different type of accesses:

- Random access, where each index can be queried with equal probability, completely at random and without any bias in the query selection.
- Monotonic access, queries are sorted in increased order, this makes queries more space local, property that can be exploited by one of our implementations. This is the access that we valued the most since the query set given by Ebay represented such access and must be optimized as much as possible.
- Full vector traversal, all elements are accessed sequentially from the first to the last index. In such scenario the vector is read completely by the implementation.
- Full vector decompression, the entire vector is decompressed at once on one single call.

The access patterns tested are not random; they were deliberately chosen with specific purposes in mind. Each pattern represents a useful type of access that occurs in real-world use cases, which will be listed in the following

paragraph to provide a more concrete sense of where our implementation can be applied. In particular, this demonstrates the finer-grained utility of the lightweight compression scheme, both as a standalone library and within the columnar store developed in this work.

By starting from the complete random access, it can be used in a lot of different scenarios. Remaining in the context of forward indexes and search engines, random access can be required to access arbitrary positions in an index to retrieve specific document ids that are relevant to the user. Another clear use case can be for example the implementation of a find in page algorithm for pages or similar data.

Monotonic access, can have multiple use cases and it is the most valued one in this case.

The first use case, serving as an example, comes from the use of posting lists in combination with skip pointers. Consider a query that requires elements from two posting lists to be connected using an AND conjunction. In such a scenario, one posting list may be significantly less dense than the other. Without skip pointers, the query evaluation would require iterating through the entire denser list, reading many values that are guaranteed not to appear in the sparser one.

To avoid such a waste usually skip pointers are stored in memory. Skip pointers allow to know the presence of an element in the sequence and allow to skip to its position. Storing this extra metadata reduces the amount of comparison that are made during the merging.

Increasing or decreasing the number of skip pointers changes the performance gains obtained by their use.

This kind of optimization is used a lot in real world search engines and is crucial for a lot of different queries. The monotonic accessing comes into play when the posting lists, which are implemented using inverted indexes, return ids as a result. These ids are sorted, because posting lists are sorted, and so they return sorted results spaced between them.

Having monotonic access optimized, allows to have fast performance on these types of queries that are of fundamental importance in search engines.

In general every operation that requires forward accessing, but does not inherently require to access all elements benefits and can exploit our implementation.

Another type of access that was considered during benchmarkings was full traversal. One field that benefits and uses full traversals are analytics. For example, in order to calculate an average or a maximum, all elements that were previously compressed must be touched.

It is not always possible or beneficial to directly decompress the whole vector. In fact, based on the platform, the uncompressed dataset might not completely fit in main memory. Such case is the worst, since it would require accessing the disk by using paging, which can lead to really high access times. On top of this, as it will be shown and as it was discussed, storing

and accessing data compressed as columns can be faster than accessing the same data uncompressed.

Another example for complete accessing is re-ranking scores, where some algorithms can require full traversal of the data in order to re compute the ranks and the scores.

The last benchmark, for the accesses to the data, is the full decompression. The full decompression is mandatory since not all algorithms are designed to work with compressed data.

Being able to pass the whole vector decompressed to an algorithm that requires it, avoids the need to implement compressed data handling inside of it. An example can be machine learning based ranking systems, that work only on decompressed data.

Another case happens on columnar databases where the same data is queried a lot of time. In such cases having the whole data decompressed can be beneficial compared to accessing it over and over. If the CPU is stressed, switching the workload to a more memory intensive, by avoiding constant and repeated decompression, can be beneficial.

The aim of these benchmarks will be to prove why and when compression is beneficial for random access. The idea is to show that:

- Memory usage is reduced, attribute that can be crucial for certain scenarios where memory has a high value. Memory usage reduction also leads to smaller data structures that fit easier in cache.
- Improved cache locality, packed data increases the chance that relevant values are already in loaded cache lines, which means that a single cache line brings more information compared to an uncompressed one.
- Better CPU utilization, memory transfers dominate access time usually, when this happens the CPU stays idle waiting for data to come. Compressing the data can reduce this phenomenon by transferring more significant data that requires the CPU to decompress.
- Faster access, in some scenarios the access time is reduced by the above mentioned properties of compressed data. In particular our implementation shows benefits in sequential or clustered queries that can exploit higher locality and then better cache utilization.

The following Section also aims to reinforce the arguments presented in Section 5.4, further proving why bit-packing was the most reasonable choice in this scenario. The main advantages given by bit packing, that will later justify some benchmarks results, will be explained below, in order to give a better idea of what properties come into play when doing these benchmarks. The first main property that bit packing has is the fast random access. This comes from how numbers are encoded, it in fact requires just some simple

bit masking to retrieve the actual uncompressed data. This is the main characteristic that drove us to chose it as compression method, in order to get the best access time possible.

Another factor that can help in some of the above mentioned access patterns, is the possibility of using SIMD operations. The library that was wrapped made extensive use of SIMD, allowing us to get an advantage on batch decompression and accesses that required multiple decompression at once.

Given that access time was the main focus, maximizing the compression ratio was also in the requirements. Bit packing allowed us to have this high random access performance, without sacrificing the compression aspect on the target datasets.

The last two reasons that encouraged us to use bit packing were the easy implementation and the adaptability that it can have. The work was guided by the perspective of building an extensible tool.

The idea is that bit packing is just the first compressor which serves as the case study to prove the benefits coming from compressors that are lightweight, in an optic where multiple compressors can be implemented in the hybrid index, in order to allow it to have such gains on multiple types of distributions.

7.3 Bit packing benchmarking against existing random access alternatives

This Section will compare the random access implementation of the blocked bit packing and blocked bit field against valid alternatives that lie at the same implementation scope, using the same hardware described in 7.5.

All of the compared implementations are lightweight and suitable compressors/data structures that can fit the use case where ours reside.

The first implementation used for comparison, comes from a professor Rosano Venturini GitHub [34] repository.

This repository exposes several tools that come handy when writing Rust code. In particular the `BitField` structure is the relevant one for these comparisons.

A bit field is a data structure that allows to store integral types values using a fixed number of bits. This type of data structure is also available in C and C++. It requires integral types in order to have a stable amount of bits to work with, during masking and similar operations.

Having a fixed amount of bits for each element gives several advantages and allows to store the sequence of values as a sequence of bits. One of the advantages is packing. The data structure stores each value with a given length, which is the field width, this width represents the number of bits used to represent each value. This makes the data structure similar to bit packing and a good comparison for our tests.

The data structure allows for random access as well. Random access is pretty easy to implement, since each value is just a sequence of bits at a given position in the `BitField`, this position is given by the product of the index times the bit width. The value extraction procedure is just a mask operation using, again, the bit width since the value to extract is just the sequence of bits starting from the offset plus the bit width.

The formula:

$$\text{offset}(i) = i \cdot w$$

$$\text{value}_i = (\text{BitField} \gg \text{offset}(i)) \& ((1 \ll w) - 1)$$

where:

- i is the index of the value (starting from 0),
- w is the bit width of each value,
- $\gg$ denotes the bitwise right shift,
- $\&$ denotes the bitwise AND,
- $(1 \ll w) - 1$ creates a mask of w ones to extract the value.

The `BitField` shift, is not how it is implemented but just a logical representation of what the operation to be performed is. The data that is used to do the bit wise AND, is instead just a slice of the compressed data that is shifted in order to only keep the relevant values.

Some of the other comparisons were made with the `compressed-intvec` crate [16] implemented by another University of Pisa student.

This crate allows to represent vectors of integers efficiently in memory. The library offers different compression algorithms based on the user's needs. Obviously each compressor has its own advantages and disadvantages based on the data distribution.

According to the GitHub repository description:

“...It offers two complementary data structures: `FixedVec`, which uses fixed-width encoding for $\mathcal{O}(1)$ mutable and atomic access, and `IntVec`, which uses variable-length instantaneous codes for high-ratio compression and amortized $\mathcal{O}(1)$ random access.”

The combination of random access and high compression ratio makes this implementation particularly suitable as a benchmark for comparison.

From this repository the Gamma encoding, Variable Byte and the fixed width encoding were used for the benchmarks. `vector`'s APIs, but makes use of run length encoding under the hood.

To better understand the benchmark results, it is important to briefly outline how the considered compression schemes operate.

Gamma encoding represents an integer in two parts: a unary prefix that encodes the length of the binary representation, followed by the actual binary digits of the number without the leading 1. This makes it very efficient for distributions with many small values which is the exact use case where it will be tested. The benchmarks are obviously conducted on datasets that allow a good compression and a good scenario where bit packing can be used. This is the reason why Gamma was included in the comparison.

Variable Byte encoding instead splits integers into chunks of 7 bits, each stored in a byte, with the most significant bit serving as a continuation flag to indicate whether more bytes are needed. In this way, small numbers can be stored in a single byte, while larger ones expand to multiple bytes, balancing compression and decoding speed. Variable byte was another candidate considered during the design phase. The issue with this compression scheme though lies in the overhead required to implement random access which will be analyzed and thoroughly explained later when results are presented.

Fixed width encoding performs and is implemented as a BitField and ends up having similar values to the one implemented by professor Venturini.

Run length encoding, encodes runs of identical values by specifying the number and how many times it is repeated in the run. The algorithm is not expected to perform well on the dataset used, but it was inserted in the benchmarks since it was considered during the design phase.

Run length encoding is good on sorted data and is a good candidate as one of the possible next compressor to implement.

In addition to these, another implementation was developed: BitField with blocks. This solution was built upon Professor Venturini's original *BitField* implementation, which I extended to support block-based compression.

The implementation was made since BitField becomes really inefficient in datasets distributed as power laws. So the idea is to have the same advantages obtained in the Bit Packing implementation also on the BitField data structure.

This implementation allows to define the size of the blocks used to encode. There is the constraint though of only having power of two block sizes. The reason behind this is that, during the block calculation, modulo operations and multiplications can be reduced to bit shifts.

This type of optimization allows to give a substantial increase in access time when querying for an element.

The various compressed data structures were compared with the standard vector in order to show the increased speed in random accessing, when the access is memory bound, rather than CPU bound.

7.4 The dataset

Most of the benchmarks were made on real world dataset. In particular the following sections, if not specified otherwise, will be based on data gathered from a Twitter dataset [25].

The dataset was cleaned in order to obtain the relation of follow between users. This means that the data used in the benchmarks represents the number of followers each user has.

The original dataset showed instead a list of pairs where the first represented the id of the user following and the second showed the id of the followed user, sorted by the id of the user being followed in ascending order.

The cleaned dataset is build on top of this. A Rust file was written in order to extract such information. The operations made are trivial, the pairs are scanned and for each followed id, the number of rows corresponding to it are counted.

After all of the rows for an id have been counted, the resulting number is reported in a `csv` file, the number will obviously represent the number of followers the user has.

Different datasets have been used during the process of benchmarking, but some of them were not big enough. In order to test the cache performance, datasets were required to be bigger than the L3 cache.

Most of the dataset initially used did not satisfy such requirement. The current one though is large enough to satisfy this requirement as it will be seen in the benchmark dedicated to memory consumption.

Another requirement for the dataset concerned the distribution of its values. Bit packing should be considered viable only on datasets that are composed exclusively by small values. The blocked version enhances the compressor by allowing it to be used also on power law datasets where some outliers are present.

This type of power law distributed datasets were used to test the different compressors. There is not much sense in testing the implementations on datasets that do not fit these requirements since these are preconditions to have the gains that were discussed before. On top of this, a lot of natural phenomenon distribute on power laws, which are the most common case where bit packing is the best choice given the requirements.

Another relevant use case involves datasets that contain only small values. In such scenarios, however, the benefits are limited, since relying on the smallest primitive type already reduces memory consumption to near-optimal levels, making the use of bit packing largely redundant.

Figure 1 represents the distribution of the dataset values using a linear scale. The x axis represents the rank, represented linearly, while the y axis represents the frequency associated to the value. This means that the highest value on the y axis is the maximum frequency registered among the dataset

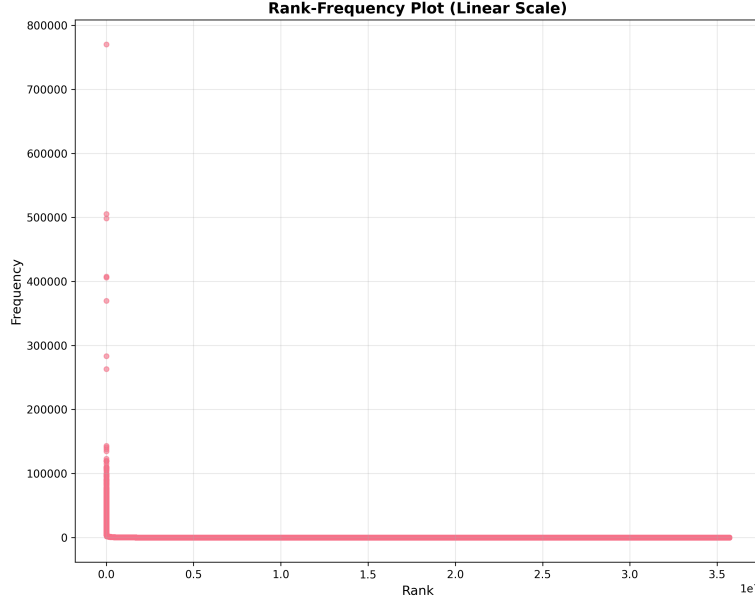


Figure 1: Rank-Frequency Plot (Linear Scale) of the dataset

values (basically the highest value in the column).

This Figure shows that values are mostly small, since the majority of the dots have a value that goes below 10000. Only a few are higher than that, these values are the outliers that make the use of standard bit packing without blocking inefficient.

Figure 2 shows the same information, but using logarithmic scales on both axes. This transformation spreads out both the most frequent and the rare items, making it easier to observe the heavy-tailed nature of the distribution. The red line represents a fitted power-law model, where the slope indicates how quickly frequency decreases with rank, and the R^2 value quantifies how closely the data follows the power-law behavior. A nearly straight line on this log-log plot confirms that the dataset exhibits strong power-law scaling, with a few very high-frequency items and a long tail of low-frequency values. Figure 3 shows the Complementary Cumulative Distribution Function (CCDF) of the dataset on log-log scales.

The x-axis represents frequency values, while the y-axis shows the probability of observing a value greater than or equal to each frequency. The red line represents a fitted power-law model, where the slope indicates the decay rate and the R^2 quantifies the goodness of fit. Linear behavior on this log-log plot confirms the heavy-tailed, power-law nature of the dataset, emphasizing the distribution of rare high-frequency items.

Finally, Figure 4 represents how the dataset values are distributed using a standard histogram.

The x-axis represents the frequency values, which are grouped into discrete

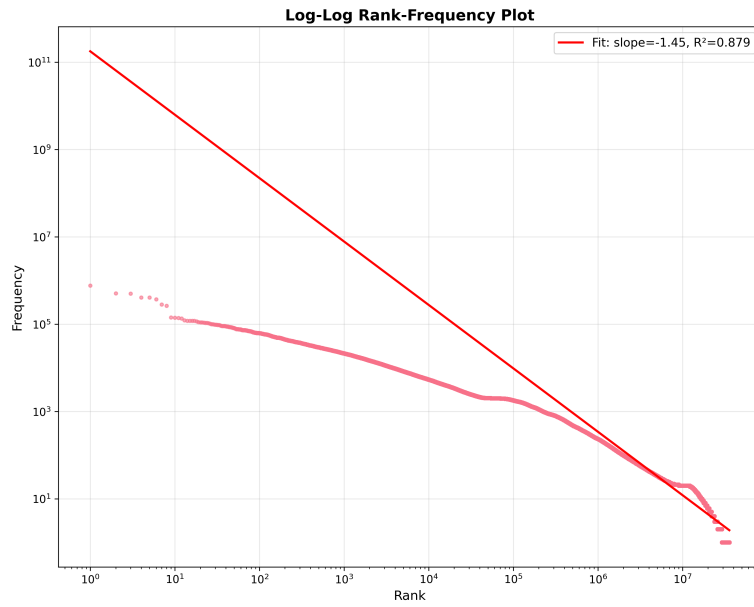


Figure 2: Rank-Frequency Plot (Log Scale) of the dataset

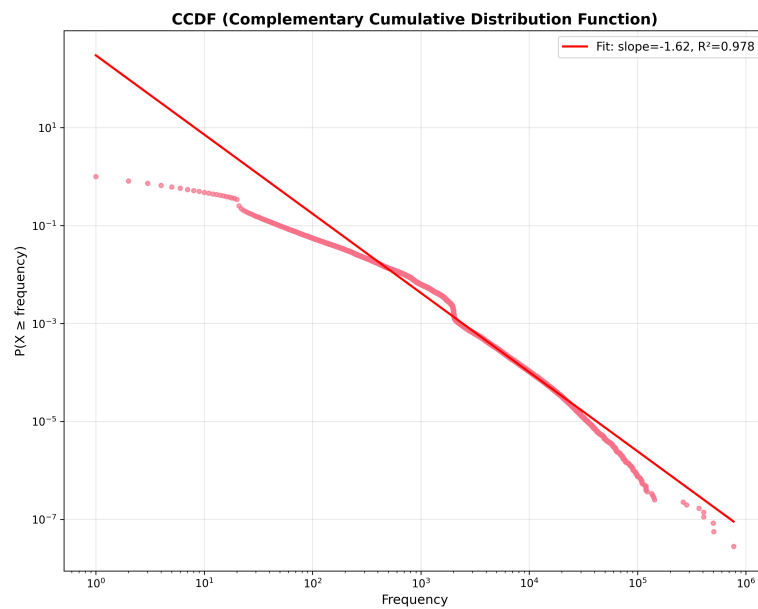


Figure 3: Complementary Cumulative Distribution Function of the dataset

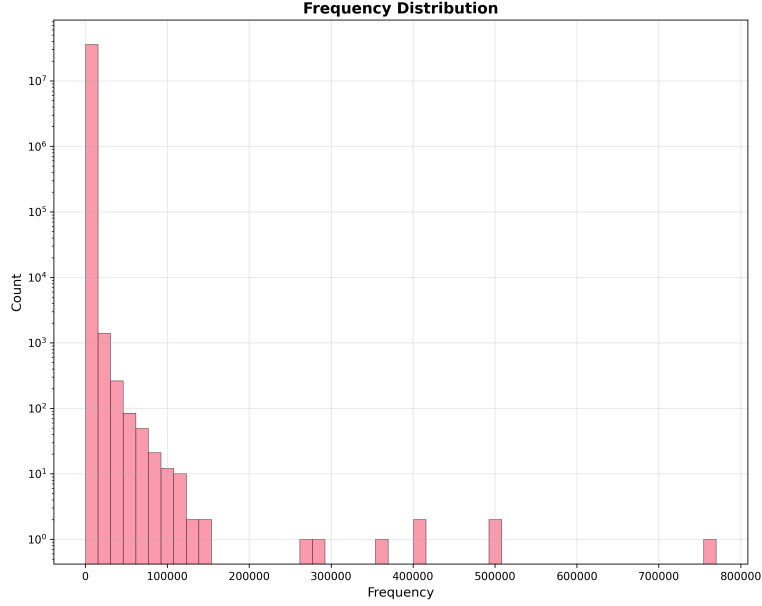


Figure 4: Histogram representing the frequencies for ranges of values

buckets. Each bucket contains all values that fall within its range, effectively segmenting the data by magnitude. The y-axis, on the other hand, indicates the number of values contained within each bucket, showing the concentration of items at different frequency levels.

In a power-law distribution, the majority of values are small, resulting in the leftmost buckets being significantly higher than the rightmost ones. Only a few values are very large, forming a long tail with relatively low counts.

The Twitter dataset used for the benchmarks clearly follows this behavior. As observed, the highest bucket is the first one, followed sequentially by the second and subsequent buckets, with the counts decreasing overall. Interestingly, the last three buckets show a slight increase compared to the preceding ones; however, this minor deviation does not alter the overall power-law distribution, which remains evident.

This characteristic, with a large proportion of small values and a few extreme high-frequency items, perfectly aligns with our intended use case, making the dataset suitable for evaluating compression schemes such as bit-packing and blocked encoding.

Another important aspect of the dataset is its size.

The current dataset contains 41 652 231 values, resulting in a CSV file of approximately 98 MB.

Considering that the L3 cache of the processor is only 16 MiB, this dataset will not fit entirely in cache, even with the most efficient compression techniques.

This property makes the dataset particularly well-suited for benchmarking compression schemes, as it ensures that memory access patterns and caching effects can be realistically evaluated.

7.4.1 Impact of Bit Packing on memory usage

The first benchmark executed regards memory consumption. The first advantage that comes out from blocking, compared to standard bit packing compression, is in terms of memory usage, especially in this kind of datasets like power laws.

Table 1: Comparison of compressed data sizes for different block sizes

Block Size	BitField	BitField Blocked	Bit Packing	Size Reduction
128	104 130 632	55 395 596	55 396 204	55%
256	104 130 632	58 213 852	58 215 052	53%

Table 1 shows memory consumption between the bit packing implementations.

Each row represents the block size used for the encoding, while the next three columns represent the various implementations along with their memory usage.

The last column instead shows the size reduction that the blocked versions bring compared to the standard non blocked BitField version.

As the table shows, in this particular dataset, increasing the block size just brings extra overhead, instead of bringing extra reduction.

Tweaking block sizes can affect the compression ratio obtained. Having smaller blocks, increases the extra overhead brought by blocking, the reason comes from the increase of the prefix sum size. It can also happen that, increasing the number of blocks, increases the maximum sum stored in the prefix sum, to the point of requiring a larger type to represent its internal values. A larger type brings higher space consumption in the overall count for the overhead.

On the contrary, bringing larger blocks, reduces the overhead by reducing the magnitude of values stored in the prefix sum, but increases the impact of outliers during the compression. An outlier in a smaller block, will force the encoding of less blocks, compared to an outlier present in a larger block. The current Bit Packing implementation works with blocks of either 128 or 256, while the blocked BitField implementation works with any power of two.

Bit Packing is restricted to only 128 and 256 since it is based on the Bit Packing library, but one considered enhancement to the implementation was allowing for more block sizes.

BitField works only with powers of two since such constraint allows to optimize blocks calculations along with modulo to simple bit shifts, which costs

less CPU cycles and increases access time.

Going back to Table 1, results show that the blocked implementations, bring overhead by storing the prefix sum, but allow to get high gains in the overall memory consumption.

On both block sizes both of the implementations use almost half of the memory compared to the standard non blocked implementation. This values were all calculated with the use of the Mem Dbg crate [37]. The crate allows to recursively calculate the amount of memory used by structs, by either implementing their defined traits, or by deriving them using macros.

Table 2: Comparison with other compression implementations in terms of size and reduction.

Implementation	Block Size	Size (bytes)	Reduction (%)
GAMMA	—	36 566 552	89%
VBYTE	—	48 002 216	86%
BitField Blocked	128	55 395 596	83%
Bit Packing	128	55 396 204	83%
BitField Blocked	256	58 213 852	82%
Bit Packing	256	58 215 052	82%
BitField	—	104 130 632	69%
RLE	—	621 254 104	−87%

The same comparison was applied to the other compression schemes that were considered to implement. Table 2 shows the results obtained.

In this case, rows represent the compression scheme applied, the blocked versions have two entries, one per block size. The first column represents the block size, followed by the absolute memory usage and the reduction obtained by its application. The reduction is calculated against the uncompressed vector.

As it can be seen the best compression results are achieved by Gamma and Vbyte, directly followed by the blocked implementations. The gains obtained are of at most six percent, which, as it will be shown later, are not enough to justify the performance trade-off in terms of access time.

Run length encoding performs the worst and ends up having more metadata than the compressed achievements. This comes from the fact that data is not sorted by frequency. Sorting by frequency is a niche case, this is one of the reasons that lead us to avoid implementing it for the hybrid index compression.

On certain scenarios it can be a good choice, but if the requirements have for example lexicographic sorting, then it makes no sense choosing such a scheme compared to the other presented here. Even assuming sorted data, the compression achieved would probably not be worth the cost paid on random access, since bit packing and other compression schemes, benefit

from sorted data.

For example, bit packing on sorted data ends up having the minimal number of polluted blocks by outlier, which is the best case scenario. Run length encoding increases its compression performance, but has a major trade-off in terms of random access that will be showed in a later about access benchmarking.

7.4.2 Monotonic access

The first type of access that has been put under benchmark is monotonic access. As it was presented earlier, this type of access produces queries that are ordered in the sense that accessed indexes increase, starting from the smallest going to the largest.

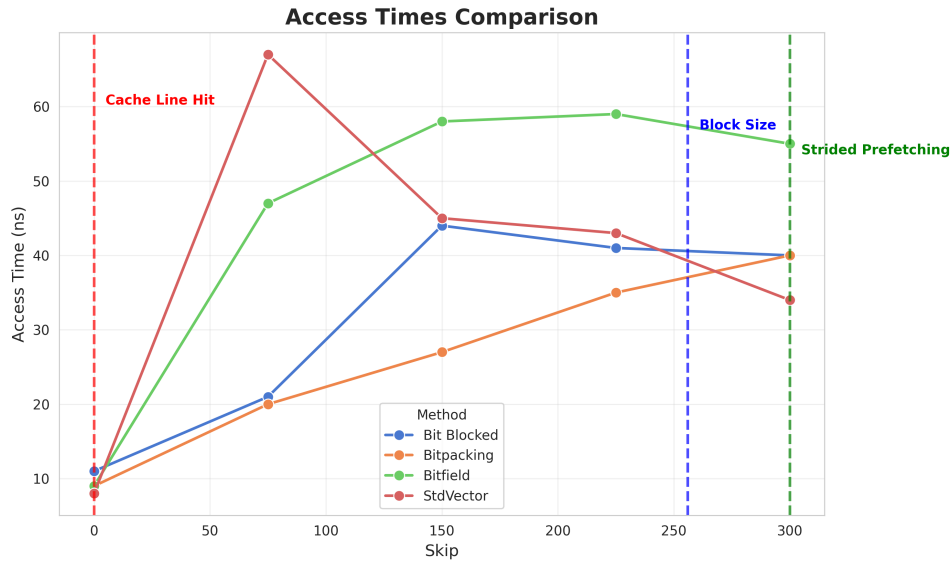


Figure 5: Monotonic access comparison

Figure 5 shows the access times for this type of access, limited to the implementations that achieved the fastest performance.

The chart shows on the y axis the nanoseconds took by each implementation on average, while the x axis shows the minimum spacing between two consecutive accesses. The queries were generated with a particular method that will be now described.

During the benchmarking, that will be showed later, with complete full random access, it was observed a particular behavior on the BitField implementations. Both the implementations were able to completely prefetch the next accesses, during the previous one. This resulted in insanely high access speeds even on datasets insanely high, with random accesses.

This phenomenon will be described again later, but it was necessary to introduce it, since the same was observed on this monotonic accessing. What happens here though, is not simple prefetching, but what is called strided prefetching.

This type of prefetching, which was observed being more aggressive on an Intel CPU compared to a newer AMD one, searches for patterns during memory loading.

Whenever the prefetcher is able to find a pattern, if it gets a hit, it will load the next address that it expects from the pattern it recognized.

For example, if the prefetcher spots a pattern where addresses at a distance of k are required multiple times, if an address a hits, then the prefetcher will immediately load the address $a + k$. For this analysis this is crucial since it can invalidate measurements, especially because real world work loads are not guaranteed to be this predictable and the CPU might be busy doing something else, not allowing for this prefetching to be applicable.

The more deterministic the access to the data structure is, the easier it is for the prefetcher to kick in and invalidate the measurement for real world scenarios. This phenomenon does not happen in other implementations such as Bit Packing blocked.

The reason lies in the fact that in a chain of queries made to the Bit Packing blocked data structure, the next query behavior depends on the previous one.

For example, if the current query n hits the current block, the next query will search inside the current cached block. If the current query n does not lie in the current cached block though, the next query will have to look inside the block that has been decompressed by the previous one.

This dependency prevents some types of prefetching from being this aggressive, making measurements more realistic.

Based on this fact, the generation of the monotonic queries tries to create a dependency between the different accesses even for BitField, where there is no such dependency naturally.

BitField in fact has only to do some masking and bit manipulation when queried, there is no dependency at all between accesses. Queries are generated starting from a random index in the vector for each run, then a constant defines the minimal spacing between queries. The smaller the constant, the more dense the accesses will be.

The dependency comes from the fact that when the query is generated it is calculated as:

$$i_{t+1} = (i_t + s + v_{i_t}) \bmod n$$

where:

- i_t is the index accessed at step t
- s is the spacing constant

- v_{i_t} is the value stored at position i_t
- n is the length of the vector

The idea is that, in order to calculate the next access the value extracted from the previous query is required. This brings some implications though, the queries for example are not exactly evenly spaced since the value that is added varies based on the value that was present in the compressed array. This makes the access not exactly spaced, but, since the distribution is a power law, accesses will tend to have distances not too much larger than the spacing constant.

It can obviously happen that a big value is queried, leading to a RAM access, but it is amortized by the high number of queries and the low probability of hitting a high value.

The modulo operation is there to avoid index out bounds accesses.

This method though is not enough, since the prefetcher is smart enough to still hit some of the accesses with the preloading, leading to a lower bound for the BitField strategies analyzed. The larger the spacing gets, the more this mechanism comes into play, since the patterns are less affected by the small variance introduced by a smaller value compared to the overall spacing, making the prefetching results more visible.

The good thing though is that the prefetching starts performing well on spacing that goes out of the scope of the BitPacking blocked, allowing to still have relevant measures for the use case.

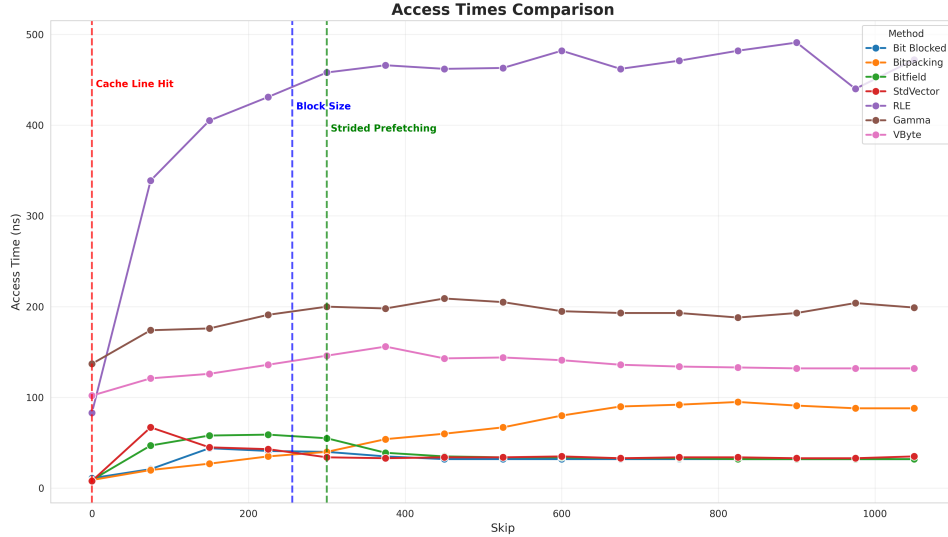


Figure 6: Monotonic access comparison with all compressors

Figure 6 shows the same comparison given by Figure 5, but with larger spacing and with all the implementations compared. The results are presented

using two figures: the first highlights the implementations with the best performance to make their comparison clearer, while Figure 6 provides the overall picture.

Let us now describe the results showed by the two Figures. When queries are really close to each other, there is little to no distance between the different implementations. This happens because, when a query is executed, the whole cache line from the access is loaded. Almost all the cache line is used, leading to a high number of cache hits, which result in fast access time for all implementations with just a few nanoseconds of difference. The red vertical line on the chart shows where this phenomenon happens.

This first dense query pattern shows though that some implementations have higher CPU overhead compared to bit packing. Non bit packing implementations are all slower than the one that are bit packing based then proving our point in choosing it over others.

Watching the immediate next measurement a spike in the access time is measured. This happens because there is no clear recognizable pattern in accesses for the prefetcher, since the accessed value can change by a lot the next query indexed and the cache line is too small to give enough cache hits. Here it is clear that the implementations that make use of blocks are faster thanks to their smaller memory transfers.

The other measurements keep showing that, as long as the spacing is reasonably close, or lower, to the block size, the blocked bit packing implementation is faster and more convenient. The reason comes from the fact that the decompressed cached block, is big enough to fit in cache, right after it is decompressed.

This means that, since accesses happen one after the other and cache is not polluted between them, a cache hit for each consecutive access will happen. This brings no decompression overhead and no extra memory transfer in cost, property which other implementations do not have. Also, as it will be showed later, having the decompressed block allows to have word aligned operations, which are much more cache friendly, compared to not aligned ones.

For what concerns the other compressors, the one performing the worst is RLE. The main reason is that the dataset is not sorted, as the requirements stated, and that RLE inherently introduces overhead during random access. To retrieve the value at a given position, RLE cannot directly index into the compressed representation. Instead, it must determine in which run the queried index falls. This is typically done through a binary search over the run boundaries, followed by a step of decoding to compute the exact position. While binary search reduces the lookup cost from linear to logarithmic in the number of runs, it still adds a significant overhead compared to schemes like Bit Packing, where the position of each element can be computed directly by arithmetic operations. Moreover, if the dataset has many short runs, the

number of runs increases substantially, making the binary search step even more costly. This means that each access for RLE in this scenario has a $\mathcal{O}(\log n)$ cost for each access, while bit packing remains $\mathcal{O}(1)$.

It is important to notice though, that if the dataset is RLE favorable, binary search is applied to a small vector, making it reasonably fast for real world usage.

Gamma and VByte are slower as it is expected for variable-length encodings. Differently from bit packing, having variable-length encoding leads to higher compression, as previously shown, but the trade-off comes at the cost of access speed.

The main reason is that decoding a variable-length integer requires extra work: in the case of VByte, multiple bytes may need to be inspected until the continuation bit indicates the end of the current value; in the case of Gamma, a prefix of unary-coded bits must be parsed before the actual value can be reconstructed. This introduces branching and bit manipulation in the decoding path, which increases the number of CPU instructions per integer and often results in branch mispredictions.

Furthermore, variable-length formats hinder the use of SIMD instructions, as each integer may span a different number of bytes or bits, breaking the regularity required for vectorized decoding. Finally, random access is particularly penalized, since finding the n -th element requires scanning or indexing metadata to locate the correct offset, while bit packing allows direct computation of the position through simple arithmetic. These factors combined explain why Gamma and VByte, while space-efficient, show lower access performance compared to block-based bit packing.

The implementations are even faster than the standard vector as shows in Figure 5. The reason is again behind cache locality and the better use of memory made by the compressed versions of the data structures.

7.4.3 Full Vector decompression performance

Another access where the blocked Bit Packing shines is the complete decompression of the whole vector.

Implementation	Time (ms)
Bit Packing	49
BitField (standard)	142
Blocked BitField	147
RLE	213
VBYTE	383
GAMMA	474

Table 3: Execution time on complete vector decompression.

Table 3 shows the milliseconds required by the different compressors to return an owned decompressed vector from the starting compressed one. This is another favorable scenario for blocked Bit Packing, since it can leverage SIMD instructions to decompress complete blocks faster compared to the other implementations.

For these measurements, blocks of 256 integers were used, leveraging the AVX2 SIMD instructions set. By using this instruction set, the implementation is able to decompress 256 integers per call, leading to faster decompression speed.

This Table shows almost no differences between the blocked version and the normal BitField implementations, the reason comes from the fact that the process is CPU bound and blocking just brings overhead for such scenarios. RLE performs relatively better in this scenario because it can generate the decompressed values on the fly, without the need to read and decode each individual integer separately. This approach makes RLE particularly suitable for workloads that require sequential decompression of the entire vector, as the cost of reconstructing runs is low and the CPU can efficiently produce values in bulk.

In contrast, variable-length encodings like VByte and Gamma require additional computations for each value. For VByte, the decoder must inspect multiple bytes until the continuation bit signals the end of the integer, and for Gamma encoding, the unary prefix must be parsed before retrieving the value. These extra steps introduce branching and bit-level operations that accumulate across the entire vector, making full decompression slower compared to RLE or Bit Packing.

7.4.4 Accessing all indexes sequentially

The last type of access where blocked implementations are faster is a special case of monotonic access, where all the indexes are accessed ascending. Basically the accessing starts from the first index and queries all of them sequentially.

Implementation	Time (ns)
Blocked Bit Packing	11
BitField (standard)	13
BitField Blocked	15
VBYTE	32
GAMMA	48
RLE	82

Table 4: Execution time in nanoseconds for different compression schemes on complete access.

Table 4, shows the performance on all the implementations on the mentioned type of access.

As expected values are really low, given the high cache locality offered by this type of access, but the blocked version are still able to perform better. This comes from what was presented before, less memory transfers and better cache locality allow to achieve such a result. On top of this, the bit packing version, achieves an even better result compared to the BitField one, since it makes fully use of the whole decompressed block.

Even the other implementations manage to achieve good access times, but are still slower than the blocked versions. For example VByte ends up being twice slower than the slowest blocked version, while Gamma is three times slower. The results obtained on all the presented accesses justify the bit packing choice. The gains that come from the other encodings, in terms of memory, is marginal compared to what is earned in terms of access time by bit packing.

7.5 Impact of Bit Packing on Cache Performance

This Section will show some analysis about the bit packing implementation and its relations with cache memory. This tests aim to show why compression is beneficial for cache and how the consume of it is reduced, leading to the before mentioned gains in terms of access speed.

Graphs used to present the results represent on the y -axis the average time to access the vector. This value is averaged across several runs, in order to avoid having strange values coming from favorable execution or particular state of the machine.

On the x -axis the graphs represent the space occupied by the overall vector, represented using powers of two.

Red lines are used to represent when a cache boundary is hit.

The blue line, represented in each graph, shows the size of the vector. The first dot of this blue line represents an input of 32 equivalent to 2^5 , each subsequent dot in the line means that the input doubles. The line associates the input size with the average access time.

The idea of this experiment is to find out when the current bit packing implementation hits cache boundaries. One of the advantages of compression at runtime, is that memory space usage is reduced, which leads to filling cache with more “dense” data which in turn leads to hitting cache boundaries later. In general, when cache boundaries get hit, the execution tends to slow down by a lot. This happens because the fastest cache level is not available any more and forces the CPU to use a slower cache level. This fetches on lower memory levels slow the execution.

This experiment shows that bit packing allows to fit more elements in cache compared to the use of a normal vector without compression.

The input data used are u32 vectors. Graphs are built on top of a csv file

that is filled by a rust binary. The final graphs are built by a python script that reads the information collected in the `csv` file and uses the `seaborn` library to represent the data.

The Rust binary that executes the code uses the same implementation that is used in the hybrid index, but the queries are generated and make use of a constant called `LOCALITY_PROB`. This constant defines how probable is that the next generated value for a query lies in the boundaries of a block from the last queried index.

This value is useful to tweak how local queries are. If all queries end up being in the same block, the implementation will be just a bit slower than a standard vector access, since the overhead will just come from some `if` statements used to check if the index is in the cached block.

On the contrary if values are completely sparse, each query will require the decompression of one block and the caching of the same.

Keeping this constant high (0.8) resembles real world use cases for this particular implementation. This tests do not exactly match the execution on the hybrid index, since queries are not forward only, case which is even more favorable given the existence of the iterator and its efficiency, which make these tests an upper bound for execution time, compared to the size of the vector. Figure 7 shows the cache usage when compressing numbers that can

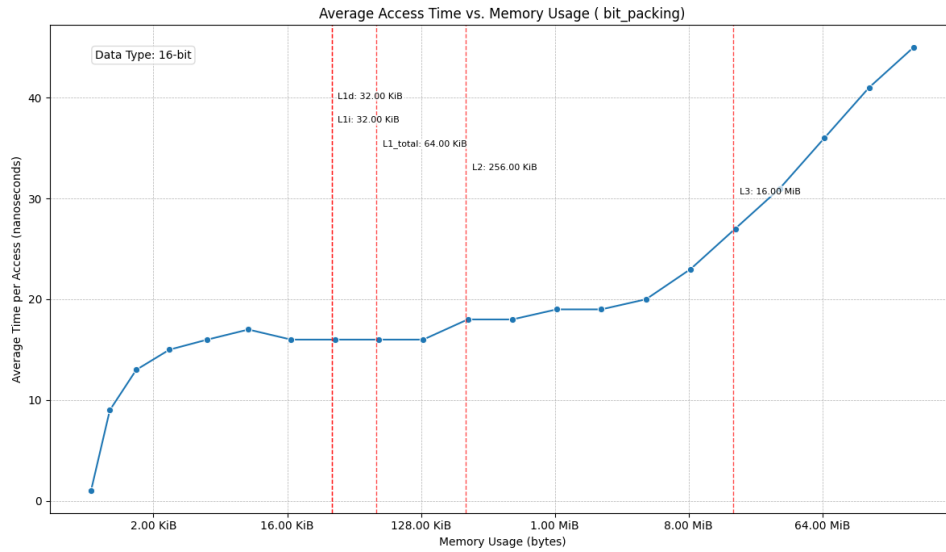


Figure 7: Cache hit performance for bit packing with values representable in at most 16 bits

be represented with at most sixteen bits.

Tests were conducted on numbers with low bits usage, given that bit packing should be used when the majority of numbers represented use a small number of bits. Cases where a high number of bits is required can still use

bit packing if, for example, deltas can be used, otherwise other compression schemes are preferred.

Figure 7 shows that access time increases close to cache levels and keeps growing as the input grows.

The increase in time access is caused by cache misses that require additional lookup compared to direct fetching from cache memory.

Ideally the red bars, and the increased access time, move further to the right as the compression ratio increases. The compression ratio is just dictated by the number of bits required to represent each number.

Figure 8 shows instead the cache usage when compressing numbers that

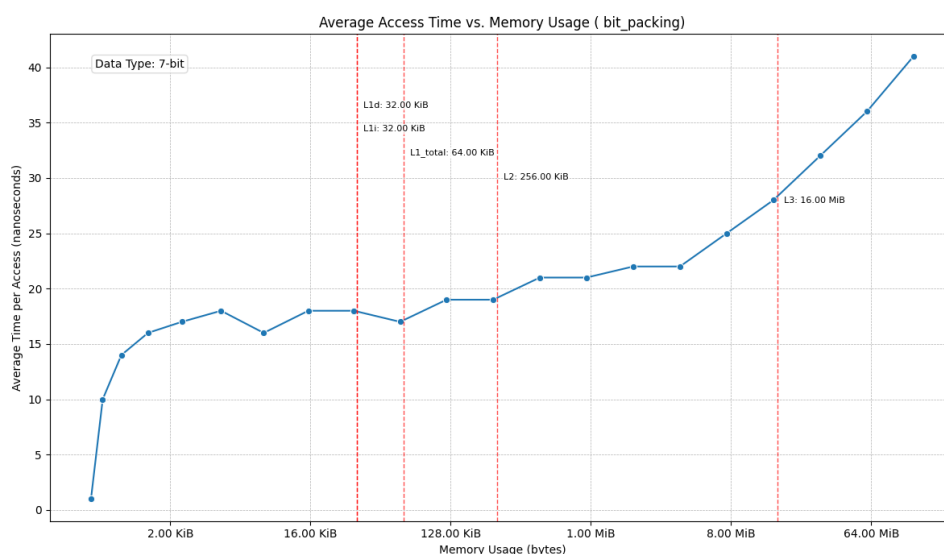


Figure 8: Cache hit performance for bit packing with values representable in at most 7 bits

can be represented with at most seven bits.

By ignoring the extra metadata that is stored to keep blocks and related information, the compression ratio in this scenario more than doubles, which should move the bars a bit more of a power of two to the right (by moving the vertical line to the right, we indicate that a greater number of blue dots are located to the left of the line).

Also run times are faster compared to previous Figure 7, which shows the advantages of having more data in cache and smaller data to move around. Figure 9 shows the cache usage when compressing numbers that can be represented with at most two bits.

This case is highly favorable for this compression scheme, since it gives a really high compression ratio. As the previous Figure 8 showed a decrease in access time, the same happens here.

The run time does not even hit the forty nanoseconds time. Also cache lines

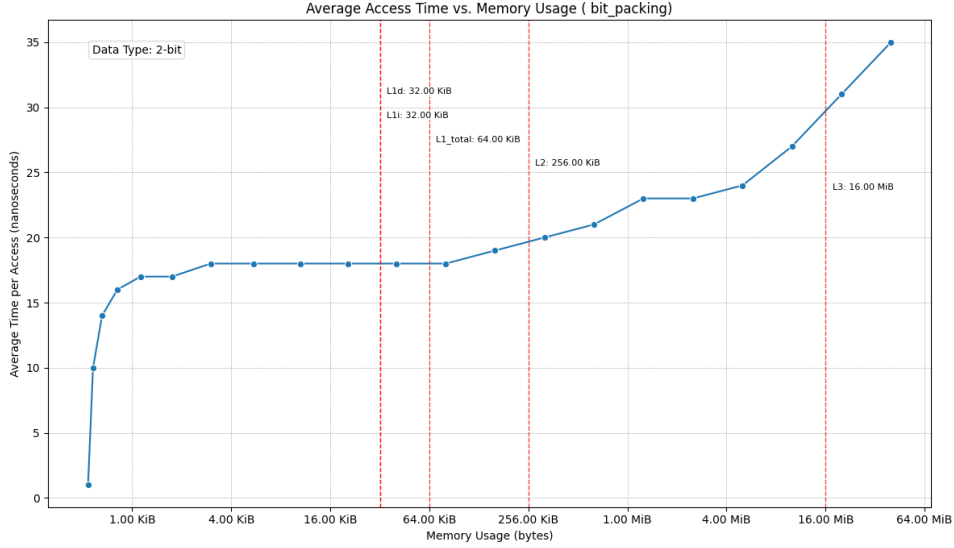


Figure 9: Cache hit performance for bit packing with values representable in at most 2 bits

move to the right compared to the previous examples, having the last cache boundary set at sixteen MiB.

Another test, aimed to show how cache is used by the blocked Bit Packing implementation was conducted. The following tests were made on the Twitter dataset presented in Section 7.4 and not on synthetic data.

Implementation	Cache References	Cache Misses	Miss Ratio
Standard Vector	155 652 341	35 819 012	23.01%
Bit Packing	179 486 971	30 042 273	16.74%
BitField	410 106 528	65 985 194	16.09%
BitFieldBlocked	384 685 791	61 608 524	16.02%

Table 5: Cache performance comparison between blocked implementations.

Table 5 compares the blocked implementations and the standard implementation. All the data was gathered using the `perf` command using hardware and software counters.

The first column shows cache references. Cache references represent all the numbers of accesses to all levels of the cache, so L1, L2 and L3 are present. In the accesses both misses and hits are counted, so it shows the overall cache usage of the implementation.

The next column represents how many of those accesses were misses, while the last column represents the percentage of misses among all those accesses. This test shows how the standard vector and the blocked Bit Packing ver-

sion reference the cache much less than the blocked and standard BitField counter parts. The Standard Vector implementation references the cache about 39%, and Bit Packing about 45%, compared to the two BitField implementations. This test was conducted on the monotonic access, since it is the most interesting use case for our implementation.

These results are the reason why Bit Packing performs better than the others compression schemes in this specific access pattern. The main reason why the standard vector and blocked Bit Packing are able to achieve such low cache references lies in the fact that both use word aligned accesses.

Word alignment is crucial, since in non word aligned accesses it can happen that a value is represented across words, which leads to values that span across different cache lines. Such behavior leads to the need to reference two cache lines for a single value, which is inefficient and leads to worst cache usage.

The alignment of the blocked Bit Packing comes from the decompressed cached block, which is represented as a std vector of `u32`, which is obviously word aligned. Although the miss rates are similar, the two BitField implementations remain slower because the higher number of references leads to more absolute cache misses, which negatively impacts access speed.

In summary, these tests show that compression on main memory can help reducing cache misses and increase cache hits sometimes. The Figures show how compressing the data allows to store more numbers from the uncompressed vector in cache if compressed. This is crucial in modern days, since CPU calculation power increased faster than how access time to main memory reduced. This means that the process stays idle waiting for data, leading to slow downs. Using compression allows to fit more relevant data in cache, which is faster in accessing, but also allows the processor to decompress instead of being idle during accessing, which can improve access performance in certain scenarios.

Additionally, proper word alignment helps improve cache utilization in these scenarios. By ensuring that data structures start at memory addresses aligned to the word size of the processor, the CPU can fetch and store data more efficiently.

7.5.1 Complete random access

This is the final Section presenting tests focused exclusively on the compression component, without involving the index implementation. Subsequent tests on the index will demonstrate that the performance benefits observed from compression alone are also significant when applied in the context of column stores and the innovative hybrid index implemented.

The tests in this Section focus on complete random access. Each query may target any index without bias or prior information, making every access

independent of the others.

Table 6: Random access times (in ns) for different implementations and block sizes.

Block Size	BitField	StdVec	BitField Blocked	Bit Packing
128	39	40	46	108
256	39	40	45	120

Table 6 shows results obtained on complete random access.

Each row represents a block size, while each column represents one different implementation, sorted by the fastest to the slowest.

Table 6 shows the strengths of the BitField implementation. This struct is even faster than the standard vector for full random access, this happens because the benchmark was conducted on a dataset that completely fits in RAM, the dimension was around 3 times the L3 cache, for this reason the majority of the accesses are out of cache and go to main memory.

BitField is this fast because the operations that the CPU has to do are minimal, compared to the time spent waiting for the data to reach the processor, this makes it the fastest.

This type of access highlights the weakness of the blocked Bit Packing implementation, which requires more than twice the time to perform a single access.

This trade-off was expected, as the main goal of the design was to optimize performance for monotonic access patterns, such as those observed in the EBay queries dataset. Complete random access was not a primary target for this data structure.

This weakness drove us to the integration of blocking also on BitField which proved to be a good idea. As Table 6 shows, the blocked version of BitField is slower, but close enough to make it a good choice on certain datasets like the one presented during the benchmarks. In fact, if the use case is a power law, as the example brought here, the cost of using standard BitField would be doubling the memory usage to gain 7 nanoseconds in access time.

This trade-off can be acceptable in certain scenarios, but if the platform values memory, then BitField blocked is a really good choice.

As mentioned briefly before, during the benchmarks a phenomenon was observed that is worth noting, as it characterizes the BitField implementations. Specifically, some unusually low access times were caused by aggressive prefetching optimizations applied by the compiler. This was strange since it happened on every dataset of every type, the registered access time would not change in any way, neither by reducing the dataset nor by increasing it. Table 7 shows the strange results that were unrealistic and did not make sense. The first implementation of the tests, iterated through a set of queries

Table 7: Access times with prefetching.

Data Structure	Size (bytes)	Access Time (ns)
BitFieldStandard	104 130 632	17
BitFieldBlocked	58 213 852	20
BitPackedBlocked	58 215 052	114
GAMMA	36 566 552	255
VBYTE	36 566 552	261
RLE	621 254 104	561

generated before time measurement. What happened was that the Rust compiler was smart enough to prefetch accesses while the CPU was decompressing the previous accessed value. This led to always have the next accessed value in L1 cache, which has this type of speed, even if the whole dataset did not fit in any level of the cache.

Several methods were tested to prevent such behavior, the reason stands in the fact that, in real world applications the compiler will never be able to predict the next access, making the benchmark useless to figure the real performance of the implementation. The first approach to fix this behavior was based on choosing the next address by using a modulo operation on the value obtained by the previous query to create a dependency between accesses.

Using a small modulo value was not enough since the compiler could still load the correct cache, since values were close to each other, lying in the same cache line. The modulo of the previously extracted value was not enough as well, given that the dataset contained mostly small values.

The final solution adopted was the use of a light weight random number generation for each query, this allowed the compiler to avoid completely the prefetching leading to the results presented in the Table. The overhead introduced by the number generation is minimal and does not compromise measurements.

The other presented implementations do not suffer from such phenomenon since the compiler might end up on branching during accesses and remains conservative on the predictions that it makes. The same happens on Bit Packing, since the access behavior, depends on the previous access.

Adding if statements inside the access, to make it artificially unpredictable, was not a viable option, since it would change the implementation behavior and would increase its access time not resembling the reality of its performance.

The `black_box` hint was used on all benchmarks, but it was not enough to prevent such behavior from the compiler. The hint was applied on both the queried indexes and on the function calls, but probably the optimization is at a higher scope and it cannot be avoided that easily.

7.6 Additional compression datasets

The compression implementations were tested on two additional datasets, confirming the expectations and giving the same results.

The first additional dataset that was used is a corpus containing word frequencies coming from the English literature, gathering texts from 1700 to 1922 [31].

This dataset gathers a large number of texts coming from the time span going from 1700 to 1922, collecting the frequencies of terms appearing on each volume. The data is divided by volume categories. The categories are fiction, poetry and drama. Each of these categories is then divided by years, which is then divided into volumes and their frequencies.

The amount of data in the overall dataset was too big for the use case, only a sub set of it was used. The subset of the dataset was cleaned and the result is a CSV file consisting of one column with the frequencies of all the words combined.

The dataset contains 17 733 650 rows, that were initially sorted by frequency. The requirements did not allow to sort by frequency, then the dataset was reordered to follow a lexicographic sorting, making it less favorable for compression, but more realistic in terms of real world usage.

The dataset was chosen because words frequencies in texts tend to follow a Zipf's law, meaning that word frequencies in natural language exhibit a characteristic power-law distribution. On top of this the other main requirement was about the size of the dataset. Initially other similar datasets were chosen, like word frequencies coming from Wikipedia pages, or GitHub stars distribution on the top 40 000 repositories.

These datasets followed a power law distribution, but they were not big enough to exceed the L3 cache size, making tests too fast, given their small size, producing miss leading results.

The other dataset used comes from Wikimedia statistics [35]. In particular the dataset used is the QRank one.

This dataset aggregates page views for each page present in all the Wikimedia projects. The dataset fits the requirements for the benchmarks given its size (28 691 759 entries) and since its power law distribution. The idea about the distribution is pretty simple, only a small amount of pages receive a lot of views, in the total count. Given the large number of pages currently active for the projects, the distribution will just have a small amount of high visited pages, while all the others will be highly compressable using Bit Packing.

The dataset format is the same of the Corpus one, one single column containing the frequencies for each page. In this case the frequency represented the number of cumulative visits for the page.

This dataset as well was not sorted as the requirements specified.

Both of the datasets exhibited the same behaviors presented in the detailed

previous Sections, proving the effectiveness of the blocked compression strategy for power law distributed datasets.

Some results are not included again given their reduced relevance compared to the scope of the work. For example, run length encoding is not presented again, given its poor results on the specific type of dataset used as a case study.

Implementation	Time (ms)
Bit Packing	35
BitField (standard)	112
Blocked BitField	115
GAMMA	366
VBYTE	425

Table 8: Execution time on complete vector decompression for the Wikimedia dataset.

Data Structure	Size (bytes)	Access Time (ns)
BitFieldStandard	107 594 152	48
BitFieldBlocked	65 744 756	56
BitPackedBlocked	68 711 616	142
VBYTE	45 717 840	256
GAMMA	51 272 576	234

Table 9: Random access times measured on the Wikimedia dataset.

Implementation	Time (ns)
Blocked Bit Packing	11
BitField (standard)	12
BitField Blocked	14
VBYTE	17
GAMMA	20

Table 10: Execution time in nanoseconds for different compression on complete access for the Wikimedia dataset.

Figure 10 and Tables 8, 9, 10 all show results coming from the Wikimedia dataset.

The dataset is slightly different in terms of size and distribution, this is the reason why results are slightly different, but still consistent with what was expected.

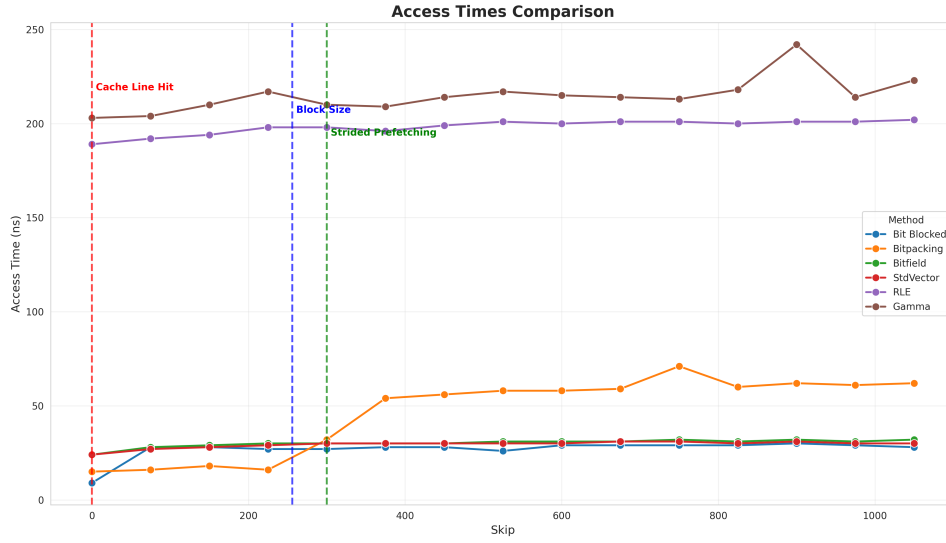


Figure 10: Monotonic access comparison on the Wikimedia dataset

Implementation	Time (ms)
Bit Packing	21
BitField (standard)	69
Blocked BitField	71
VBYTE	148
GAMMA	205

Table 11: Execution time on complete vector decompression for the Corpus dataset.

Data Structure	Size (bytes)	Access Time (ns)
BitFieldStandard	64 284 536	43
BitFieldBlocked	24 789 276	48
BitPackedBlocked	24 790 412	99
VBYTE	19 929 776	148
GAMMA	10 660 856	194

Table 12: Random access times measured on the Corpus dataset.

Implementation	Time (ns)
Blocked Bit Packing	11
BitField (standard)	12
BitField Blocked	15
VBYTE	18
GAMMA	21

Table 13: Execution time in nanoseconds for different compression on complete access for the Corpus dataset.

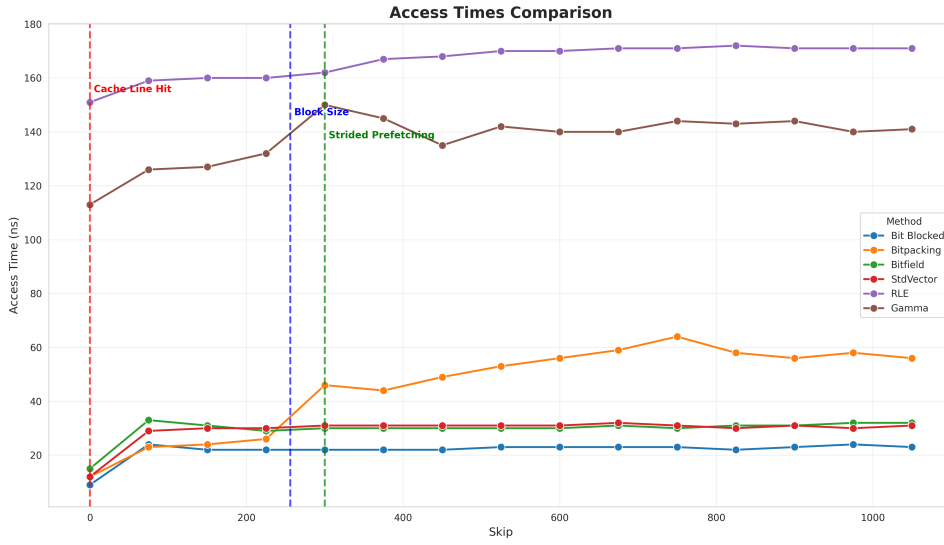


Figure 11: Monotonic access comparison on the Wikimedia dataset

The second dataset showed that compressing with blocks of 128 was a bit more favorable in terms of compression compared to compressing with blocks of size of 256. The tests were still conducted with blocks of size 256, in order to keep consistency with the previous ones and allow a more stable comparison.

Figure 11 and Tables 11, 12, 13 all show results coming from the Corpus dataset.

All results align with the expected values, the only exception stands in the results gained from the BitBlocked implementation measurements showed in Figure 11. Values are constant across the overall spacing, with consistent unexpected low access time.

The phenomenon is probably caused by strided prefetching being particularly effective on the specific dataset. The expected results should have exposed an increase after the first spacing steps, stabilizing later in the measurements.

7.7 Benchmarking Hybrid Index on Real-World Datasets

As already described in previous sections, the index was initially developed on synthetic data used to create ad hoc data for specific testing purposes. The synthetic data generation is still available, but it was not used for benchmarking, as it does not reliably provide insights that are representative of real-world scenarios. Such data can introduce biases, lack the complexity and variability of actual datasets, and may lead to misleading conclusions about system performance.

The use of TPC benchmarks was considered as well during the design of the benchmarking strategy.

TPC benchmarks are a suite of benchmarks used to standardize the process of performance evaluation for databases. Several of the resources cited in this thesis make use of the suite.

The purpose of this benchmarking suite is to unify the evaluation process by providing a standardized framework for data generation, database schema definition, and query execution.

There are different specifications defined for the suite. Each specification aims to benchmark different types of databases and their capabilities, based on their structure and use.

For our specific case, the most interesting and considered specifications were the TPC-C and TPC-H specifications.

The first aims to test capabilities of databases in a scenario of OLTP workloads. As it was discussed in the introduction 2.1, row stores are better suited for this type of workload. The idea was using this test suite for the row store portion of the index.

The second specification, TPC-H, is intended for more analytical support and it is meant to benchmark columnar stores. This specification was considered to test the columnar stored fields in the hybrid index.

The problem with using this specifications were the requirements needed to run them. The TPC suite is oriented for full DBMS rather than low level storage components, like the current implementation. The complex queries and relations defined in the schema were the main reasons why the suite was not used.

Different real world datasets were used, in order to test the implementation. The following subsections will describe the most relevant one and provide some metrics about access time and low level parameters used to profile the performance of the index.

7.7.1 Chicago Crime Dataset

The dataset used to benchmark the index implementation contains information about crimes that occurred in the city of Chicago [22]. It was obtained

from Hugging Face and provides detailed records about crimes reports happened in recent years.

This dataset was chosen for its length and the data types that it includes. Several e-commerces datasets found, which are a typical application field for the current implementation, contain few numerical values and mostly strings instead, which do not show the capabilities and advantages gained by the usage of lightweight compressors on columnar storage, which require integers instead.

To instead test column store capabilities and the bit packing compression, numerical fields, with relatively small numbers are required, property satisfied by the presented dataset.

These are the reasons that drove to the choice of using this particular dataset. The dataset contains around eight millions rows and twenty-two columns. The number of columns is smaller compared to the one used in the serialized queries, but, in order to have the same access pattern given by the requirements, they are remapped to fit the correct number of columns.

The schema includes a variety of field types, providing a comprehensive basis for testing the implementation. The key types, are summarized below.

Multiple string fields are present, this allows to test the variable fields implementation along with the new reference access for blobs and strings.

Several numerical numbers are present as well, both large and small. For example ids are large numbers that go in the order of millions, while the year field spans across the values of two thousands. Also the district field is a numerical number that contains small values which is a good candidate for bit packing compression.

The dataset also contains the location field, which can be `null`, and then be used to test the validity byte implementation.

The dataset schema was generated using the scripts developed to automate the process. As it was described, this process ties the data to Rust primitive types, but also tries to find the best type present. This means that, for example, the year field will not be represented using an `i64` like Hugging Face represents, but will rather use a smaller type that best fits the small numbers represented. This obviously assumes that the dataset is not inserting values that go out of the range of the already represented values.

Another consequence of this transformation is the compression ratio obtained from these fields, which obviously decreases if compared to the same compression applied with Hugging Faces' defined types.

After the base schema was generated, it was updated using the script interface, in order to apply compression to suitable fields and define their memory layout.

This extra manual step is required since the script does not yet infer the memory layout and the best available compression scheme. Furthermore, such decisions are typically more effective when made manually rather than

through automated processes since the decision requires external information regarding performance requirements about the field and its use cases. Below a portion of the generated JSON schema file is presented, the full version of the file is not included due to its length and redundancy.

```
1 {
2   "schema_name": "crimes",
3   "schema_id": "1",
4   "fields": [
5     {
6       "id": 0,
7       "name": "ID",
8       "type": "u32",
9       "storage_layout": "row",
10      "unique": false,
11      "nullable": false,
12      "faceted": false,
13      "compression": "none"
14    },
15    ...
16    {
17      "id": 11,
18      "name": "District",
19      "type": "u8",
20      "storage_layout": "columnar",
21      "unique": false,
22      "nullable": true,
23      "faceted": false,
24      "compression": "bit_packing"
25    },
26    ...
27    {
28      "id": 21,
29      "name": "Location",
30      "type": "string",
31      "storage_layout": "row",
32      "unique": false,
33      "nullable": true,
34      "faceted": false,
35      "compression": "none"
36    }
37  ]
38 }
```

The **District** field is a clear example of type inference that is done by the script. In this case the smallest type usable is **u8** and, on top of that, compression is used. These are just three of the twenty-two fields present in the dataset, but they show almost all of the previously described properties about fields.

Metric	Value
Number of runs	10
Number of documents	8 104 658
Number of queries	5 094 699
Number of accessed documents	28 401 509
Number of dense accesses	190 449 853
Dense fixed accesses	143 629 048 (55.44%)
Dense variable accesses	46 820 805 (18.07%)
Number of rows	8 104 658
Total access time	49.43 s
Average access time	23 ns
Total accesses	2 094 948 383
Number of queries	566 186 872

Table 14: Summary of access statistics using compressed version

7.7.2 Results on the dataset

Different metrics will be reported here to measure the implementation performance. Some indicators used in this profiling, come from the well known **perf** tool. **perf** [20] is a command that allows to access performance counters and is directly implemented in the Linux kernel. The command allows to track different important metrics like branch misses, cache hits, CPU cycles and similar metrics. Some of these metrics will be presented and compared to the previous implementation to show benefits and trade-offs of the new version against the previous one that did not include the columnar store.

Table 14 shows basic access performance of the implementation. The gathered results come from the average of ten runs executed on the before mentioned dataset. The queries set is the one provided by EBay, which accesses only going forward in documents ids and skips some documents during access.

These results come from the dataset built on top of the previously described schema. In the benchmarked implementation showed in Table 14, all compressible fields are subject to compression, while columnar storage is adopted as the underlying layout for their representation.

The aim of this first benchmark is to show the access time in a scenario where the hybrid implementation is required. In this case, both the memory layout are present in a balanced amount and allow to present the integration in a real world setting.

The results show an access time of 23 ns as an average.

The same benchmark, was executed with the same dataset and the same queries, but with row stored only indexes, results are presented in Table 15.

Table 15: Summary of access statistics using standard row version

Metric	Value
Number of runs	10
Number of documents	8 104 658
Number of queries	5 094 699
Number of accessed documents	28 401 509
Number of dense accesses	190 449 853
Dense fixed accesses	143 629 048 (55.44%)
Dense variable accesses	46 820 805 (18.07%)
Number of rows	8 104 658
Total access time	47.28 s
Average access time	24 ns
Total accesses	2 094 948 383
Number of queries	566 186 872

The resulting access time is 24 ns, showing that the compression does not affect access time at all. The main consideration that comes from this, is that the compression is as fast as the non compressed access, if not faster. The specific benchmark averages the accesses made on the compressed indexes with the non compressed row ones, making the results not only bound to the specific columnar extension made to the index.

The test does not in fact aim to test the columnar and compression implementation only, but aims to show that its integration with the overall struct does not bring disadvantages, but gains in terms of memory and access time even if minimal for the specific scenario.

As it was shown in previous sections, access gains highly depend on the type of accesses that are made and on the dataset. The benchmark was intentionally executed on a dataset that is not completely favorable to the implementation, in order to show that its use is not reduced to very niche cases, but also on a wider set of possible scenarios and datasets, granting gains both on straight up memory reduction and in access time.

The size of the dataset matters as well, increasing it can also increase the gains coming from the application of the compression.

The next benchmark conducted, was executed on the basic implementation of the index, the one storing only by row, using a single index. As Table16 shows in this case the old implementation managed to have an access time of 21ns. Again, compared to the hybrid implementation, there is no particular overhead in running the bit packing compression, on top of this some additional considerations are needed to analyze such result.

The decompression overhead, is not the only extra computation that the hybrid implementation has to make during the access to the data. In fact the row only implementation, stores data together in single rows, accessing

Table 16: Summary of access statistics using standard row version

Metric	Value
Number of runs	10
Number of documents	8 104 658
Number of queries	5 094 699
Number of accessed documents	28 401 509
Number of dense accesses	190 449 853
Dense fixed accesses	143 629 048 (55.44%)
Dense variable accesses	46 820 805 (18.07%)
Number of rows	8 104 658
Total access time	45.13 s
Average access time	21 ns
Total accesses	2 094 948 383
Number of queries	566 186 872

them using offsets, without any extra logic involved.

The hybrid implementation is built on top of the same access pattern, but has some overhead given by the extra access made to find the correct index and the match if it is either compressed or not.

The extra match brings extra branch mispredictions and all the well known problems brought by conditionals.

The extra access time brought to the implementation, in this mixed type of accessing, is dominated by branch miss prediction and the overall extra logic needed to handle how queries are redirected to the various indexes. This type of logic is unavoidable for this type of implementation, that requires to find the correct index by using extra if statements and the auxiliary data structures presented before.

The benchmark proves that using compression along with columnar storage layout brings advantages, by showing minimal overhead even with the extra logic and extra data structures involved.

The basic implementation was expected to be slightly faster on certain scenarios, since the hybrid implementation was designed to be more easy to interact with rather than the fastest as possible, given the time constraints. It is indeed possible to remove the extra logic involved to find the correct index for each query, by exposing the layout of the indexes to the user and require queries that also specify which underlying index to query.

Such approach completely removes the overhead, but breaks some abstractions that we think are one point of strength of the data structure. Hiding the index layer helps keeping the code cleaner and easier to use, maintaining the interfaces of the basic row only implementation.

On top of this it is important to notice that the set of queries is not completely leveraging the columnar access. To really get an advantage from this

access type queries should be denser in terms of accessed indexes, in order to leverage more per block caching, that still gives benefits even in the tested queries which are not optimal.

7.7.3 Columnar oriented accessing and caching benefits

The index was also tested on a more columnar friendly environment, in order to test its capabilities on scenarios where the new columnar store feature could be exploited.

The dataset on top of which the index was built was the crime dataset but with a reduced amount of fields, compared to the original one. All the fields are column storable and are:

- Arrest
- Domestic
- Beat
- District
- Year

These fields were all stored as columnar and using the bit packing compression.

The purpose of these tests is to evaluate whether the use of a columnar store improves locality and reduces cache misses compared to the row store on the same dataset, while also analyzing the resulting access times.

The following tests were conducted using the same executable files that profile the implementations. The queries used in the various tests are generated. The choice of generating the queries comes from the need of being able to choose how much local queries should be in order to see the effects on the various analyzed metrics.

The executable, that generates the queries, allows to choose how many blocks will be touched by the queries. For example, if the constant used to describe the density is set to 1.0, then the generated queries will be distributed among the blocks in order to touch every one of them. If the same constant is instead set to 0.5, only one half of the blocks will be queried, which makes queries denser and improves locality and caching.

The first test was conducted with queries touching all the blocks, which is the worst case in terms of locality and caching since it resembles complete random access. All the tests were executed using 86872 queries, which is a fraction of the actual number of access queries present in the EBay's queries set.

Metric	Hybrid	Per cycle	Row-Based	Per cycle
L1 Loads	83 584 401	120.007	38 648 797	55.490
L1 Misses	962 642	1.382	1 686 513	2.421
LLC Loads	23 806	0.034	28 377	0.041
LLC Misses	491	0.001	5 911	0.008

Table 17: Cache performance comparison between Hybrid and Row implementations on sparse accessing

Table 17 shows results obtained from the hardware counters described in previous sections and with generated queries covering all the blocks of the structure. The last column shows the row metric, but expressed per cycle. The table shows that the columnar implementation exhibits a higher number of loads compared to the row based implementation. This outcome is expected due to the way validity bits are stored in the compressed bit packing scheme. While this increase in loads represents an aspect that could be further optimized, it will be discussed in the conclusion. Specifically, revising the current storage of validity bytes in the compressed version may help reduce this metric.

In the row-based data structure, validity bytes are stored immediately preceding the actual data. In contrast, the compressed columnar version stores bits as bytes in a separate, contiguous array to achieve a higher compression ratio. As a result, accessing validity information requires additional memory operations, effectively generating two cache loads per access instead of one. This is the main reason why higher loads are observed in the current version. Other than the loads, the table shows that misses in the columnar version are lower compared to the row implementation, and they decrease as queries get more local. Misses in L1 cache are around $1.382/2.421 \approx 0.57$ which is almost an half.

Metric	Access time	Density
Hybrid	43	1.0
Row	41	1.0

Table 18: Access performance on random access

For what concerns access time, with this query density, the hybrid implementation performs almost as fast as the basic row implementation.

As Table 20 shows, the hybrid implementation is slower just by some nano seconds. Again, considering the extra logic involved, the benchmark completely proves the point and, even in the current state of the implementation which has large optimization margins, shows that the hybrid implementation brings extra features at a reasonable overhead even on complete random

access.

Metric	hybrid	Per cycle	Row-Based	Per cycle
L1 Loads	77 519 444	111.299	38 604 527	55.427
L1 Misses	970 176	1.393	1 586 968	2.278
LLC Loads	24 095	0.035	28 914	0.042
LLC Misses	695	0.001	4 465	0.006

Table 19: Cache performance comparison between hybrid-index and row-based implementations with denser queries

The same kind of tests were conducted on denser queries, which favour the compressed columnar version.

Table 19 shows instead the cache values when a constant of 0.5 is used to generate the queries. As explained before, having a constant of 0.5 means that queries are distributed on half of the blocks for the measured runs.

Metric	Access time	Density
Hybrid	35	0.5
Row	42	0.5

Table 20: Access performance on denser queries

Increasing queries locality does not give much more advantage in terms of cache misses for the columnar implementation, but improves the access time as expected. Table 20 shows an increase in access time, to the point that the hybrid implementation out performs the standard row based one.

All reported values are averaged over ten runs using the same set of queries.

Tests were also conducted in the extreme scenario where all queries accessed the same block. While these results are not representative of typical real world workloads, they provide insight into the behavior of the implementations.

The though proved that in such a scenario the row based implementation is faster. This result was expected and the reason why the row implementation is faster lies in the fact that this type of access is not memory bound, since the block lies in cache.

When the access is not memory bound, the simpler implementation, in terms of CPU operations, is always favored. The hybrid index, other than the extra branching required to find the index, needs to reconstruct the result by inserting the validity bytes for each field. This operation requires extra computation, that makes it slower in such scenario.

Once again, the current handling of validity bytes necessitates these additional computations, which could be optimized as a potential enhancement

to the existing implementation.

The same set of access time tests was conducted on the hybrid index, but without compression. As already stated in the introduction to the technologies, storing in a columnar fashion gives the advantage of locality when querying the dataset for one field only.

Without using compression, the difference between the two data structures stands in the fact that the columnar version instantiates one dense index per field. This means that the queries for the field are redirected directly to the specific allocated index that stores that column.

This structure leads to more locality, data is closer, and to faster access times since there is less data to read when extracting the values, given that the offset is lower compared to the full row. This also means that the more columns are added to the row version, the slower it becomes, compared to the hybrid implementation.

Query Density	Hybrid Col	Row
1.0	33	42
0.7	36	43
0.5	30	41
0.2	24	30

Table 21: Access time without compression comparison expressed in nanoseconds

Table 21 shows results on the same dataset, with the same method that was used for the previous tests.

This Table shows that the column implementation is always faster compared to the row one, but they converge on access time when queries get denser. Like it was explained before, for the accesses made on the compressed version of the index, the more queries are close, the less the memory layout comes in place.

The test proves the claimed effectiveness of the columnar layout on this type of access. As stated before, this test is conducted without compression. By not using compression, the previously mentioned inefficiencies coming from storing validity bytes in a different vector are not present. Also there is no reconstruction of the validity bytes along with the values, another overhead that is mandatory for the compressed version.

The only remaining overhead in this structure arises from the branch operations required to locate the index storing the columnar field. Once located, the field is queried in the same manner as in the row-based version, but with the advantage that the data for the field is more localized.

This type of configuration of the index is achievable directly by the definition

of the schema, it is just required to use columnar store on the fields and do not use any compression by specifying it in the schema JSON file.

It is important to note that the last tests show a higher access time, since queries are dense only. As it was explained before, some accesses that are specified in the Ebay's serialized queries are not implemented yet and are skipped by default. These type of queries were not removed to maintain the exact access pattern required, since having other accesses between queries even if empty, can pollute cache which leads to a result closer to what was in the requirements.

Skipping the default case one reduces the access time, that still allows comparison since the same set of queries is used on both the implementations.

8 Conclusions

The goal of the thesis was to take advantage of lightweight compression, along with blocking, in order to increase performance on random access and reduce main memory consumption.

In particular, the focus was placed on ascending monotonic random access, which plays a crucial role in real world scenarios such as search engines and related systems.

The most suitable compression scheme was bit packing, which proved its effectiveness in the use case through the presented benchmarks. The compression scheme was adapted to further optimize monotonic accessing, by using cached information to reduce access time for the specific ascending access pattern.

The solution implemented achieved the best results in terms of access time in the targeted use case, with dense enough queries.

In order to provide an alternative also for other, more generic random access patterns, a lighter implementation, more focused on complete random access was developed, the blocked BitField presented during benchmarks.

To take advantage of the compression implementation a forward index was implemented. The contribution of this work lies in the fact that there is no solution implemented in Rust at the library level, capable of storing data on either a columnar or a row layout, giving high flexibility for the implementor and high random access performance. All the other available solutions lie at a higher level of the implementation stack and usually are full fledged database systems that do not allow access at a lower level.

Moreover, existing alternatives neither make use of lightweight compression schemes nor consistently support the specific data types that our implementation addresses. The closest presented example was Tantivy, which mainly targets strings, does not support random access with numeric absolute indexes and makes use of general compression algorithms, focused on reducing the memory space consumption, rather than increasing the access time in

memory bound accesses.

Row only or columnar only solutions exist, but none of them allows one to choose between the two, limitation that is addressed instead in our implementation.

Nevertheless, the solution presents some limitations, which are not due to the underlying approach but rather to the constraints of a thesis project, where time and scope inevitably limit the extent of implementation and optimization.

The implementation is structured to host new compression schemes. The idea was in fact to create a solution able to choose between different compression schemes based on the represented data and the type of access made on it. At the time being the index only allows for bit packing, since it was the most effective as a first compression scheme, but the code is ready to be expanded. Adding a new compressor is straightforward: it only requires implementing the corresponding trait for the new compressor struct, which can then be added to the index by adding it to the dedicated enum.

The first enhancement for the implementation would be adding new compressors, which can expand the scope to more scenarios, making the index even more flexible.

Another direction for future work would be adding more traits, not only for random access, but also to categorize the newly added compressors based on their strengths. This would mean also adding new interfaces, based on what the specific compressor does better.

For what concerns optimizations, the bit packing implementation is in a good state, while some index choices can be optimized. The main inefficiencies lie in how validity bytes are stored for compressed indexes and in how queries are redirected to different indexes.

These issues come from the necessity of keeping the interfaces as clean as possible and to satisfy the query set required. The solution to avoid the redirection logic, would be requiring the user to follow the storage layout of the structure, which is feasible, but less clear.

Another inefficiency arises from maintaining a separate structure for each index to describe its fields. It is possible to avoid this by restructuring the index to only have one struct to contain the overall schema, avoiding the need of storing multiple structures in one vector, which brings some duplication.

In summary, this work has shown that lightweight compression can significantly improve random access performance while reducing memory consumption. Although certain limitations remain, the proposed implementation establishes a solid foundation and opens the way for future developments and refinements.

References

- [1] ABADI, D., BONCZ, P., AND HARIZOPOULOS, S. Column oriented database systems. *PVLDB* 2 (2009), 1664–1665.
- [2] ABADI, D., MADDEN, S., AND FERREIRA, M. Integrating compression and execution in column-oriented database systems. In *Proceedings of the International Conference on Management of Data* (SIGMOD 2006), pp. 671–682.
- [3] ABADI, D. J., MADDEN, S. R., AND HACHEM, N. Column-stores vs. row-stores: how different are they really? In *Proceedings of the International Conference on Management of Data* (SIGMOD 2008), pp. 967–980.
- [4] AL-KATEB, M., SINCLAIR, P., AU, G., AND BALLINGER, C. Hybrid row-column partitioning in teradata®. *PVLDB Endowment* 9, 13 (2016), 1353–1364.
- [5] APACHE DORIS CONTRIBUTORS. Row store table design. <https://doris.apache.org/docs/table-design/row-store>, 2025. Accessed: 2025-07-28.
- [6] ARULRAJ, J., PAVLO, A., AND MENON, P. Bridging the archipelago between row-stores and column-stores for hybrid workloads. In *Proceedings of the International Conference on Management of Data* (SIGMOD 2016), pp. 583–598.
- [7] BRAIS, H., KALAYAPPAN, R., AND PANDA, P. R. A survey of cache simulators. *ACM Computing Surveys* 53, 1 (2020).
- [8] CHAALAL, H., HAMDANI, M., AND BELBACHIR, H. Finding the best between the column store and row store databases. pp. 1–4.
- [9] FULMICOTON, AND PSEITZ. bitpacking: Fast integer compression via simd bit-packing (rust crate). <https://docs.rs/bitpacking/latest/bitpacking/>. Accessed: 2025-07-30.
- [10] GREGG, B. The flame graph: This visualization of software execution is a new necessity for performance profiling and debugging. *Queue* 14, 2 (2016), 91–110.
- [11] GRUND, M., KRUEGER, J., PLATTNER, H., ZEIER, P. D. A., CUDRE-MAUROUX, P., AND MADDEN, S. Hyrise-a main memory hybrid storage engine. *PVLDB* 4 (2010), 105–116.
- [12] HARIZOPOULOS, S., LIANG, V., ABADI, D. J., AND MADDEN, S. Performance tradeoffs in read-optimized databases. In *Proceedings of*

the 32nd International Conference on Very Large Data Bases (VLDB 2006), pp. 487–498.

- [13] IBM. What is olap? <https://www.ibm.com/think/topics/olap>. Accessed: 2025-07-28.
- [14] IBM. What is online transactional processing (oltp)? <https://www.ibm.com/think/topics/oltp>. Accessed: 2025-07-28.
- [15] LEMIRE, D., BOYTSOV, L., AND KURZ, N. Simd compression and the intersection of sorted integers. *Software: Practice and Experience* 46, 6 (2016), 723–749.
- [16] LOMBARDO, L. compressed-intvec: A rust library for compressing vectors of u64 integers, 2025.
- [17] MONETDB TEAM. Monetdb - a column store database, 2025. Accessed: 2025-07-28.
- [18] ORACLE CORPORATION. Mysql database. <https://www.mysql.com/>, 2025. Accessed: 2025-07-28.
- [19] ORACLE CORPORATION. Oracle database. <https://www.oracle.com/database/>, 2025. Accessed: 2025-07-28.
- [20] PERFWIKI CONTRIBUTORS. perf: Linux profiling with performance counters, 2025. Community wiki for the Linux **perf** tools. Accessed: 2025-08-21.
- [21] POSTGRESQL GLOBAL DEVELOPMENT GROUP. Postgresql: The world’s most advanced open source relational database. <https://www.postgresql.org/>, 2025. Accessed: 2025-07-28.
- [22] PRATHAP, G. Chicago crime dataset. <https://huggingface.co/datasets/gymprathap/Chicago-Crime-Dataset>. Accessed: 2025-08-13.
- [23] QUICKWIT-OSS. Tantivy: A full-text search engine library in rust. <https://github.com/quickwit-oss/tantivy>, 2025. MIT License. Accessed: 2025-07-29.
- [24] RAMAMURTHY, R., DEWITT, D. J., AND SU, Q. Chapter 38 - a case for fractured mirrors. In *In Proceedings of the 28th International Conference on Very Large Databases*, P. A. Bernstein, Y. E. Ioannidis, R. Ramakrishnan, and D. Papadias, Eds. Morgan Kaufmann, VLDB 2002, pp. 430–441.
- [25] ROSSI, R. A., AND AHMED, N. K. The network data repository with interactive graph analytics and visualization. In *AAAI* (2015).

- [26] RUST PROJECT. `_mm_prefetch` - rust core::arch::x86 function documentation. https://doc.rust-lang.org/beta/core/arch/x86/fn._mm_prefetch.html. Accessed: 2025-07-30.
- [27] STARROCKS CONTRIBUTORS. Hybrid table. https://docs.starrocks.io/docs/table_design/hybrid_table/, 2025. Accessed: 2025-07-28.
- [28] STONEBRAKER, M., ABADI, D. J., BATKIN, A., CHEN, X., CHERNICK, M., FERREIRA, M., LAU, E., LIN, A., MADDEN, S., O'NEIL, E., O'NEIL, P., RASIN, A., TRAN, N., AND ZDONIK, S. *C-store: a column-oriented DBMS*. Association for Computing Machinery and Morgan & Claypool, 2018, pp. 491–518.
- [29] SUNDY-LI. Strawboat: A native storage format for apache arrow. <https://github.com/sundy-li/strawboat>, 2025. Apache-2.0 License. Accessed: 2025-07-29.
- [30] TOLNAY, D. Serde: A framework for serializing and deserializing rust data structures. <https://serde.rs/>, 2025. Accessed: 2025-08-04.
- [31] UNDERWOOD, T., CAPITANU, B., ORGANISCIAC, P., BHATTACHARYYA, S., AUVIL, L., FALLAW, C., AND DOWNIE, J. S. Word frequencies in english-language literature, 1700-1922 (0.2) [dataset], 2015. Accessed: 2025-09-18.
- [32] VANDERWIEL, S. P., AND LILJA, D. J. Data prefetch mechanisms. *ACM Computing Surveys* 32, 2 (2000), 174–199.
- [33] VENTURINI, R. Base implementation used as the foundation for this thesis, 2025. Unpublished source, provided via personal communication.
- [34] VENTURINI, R. toolkit: A set of utils i often use on my rust code, 2025. Accessed: 2025-08-17.
- [35] WIKIMEDIA FOUNDATION. Wikimedia downloads: Analytics datasets. <https://dumps.wikimedia.org/other/analytics/>, 2025. Accessed: 2025-09-18.
- [36] YUAN, P., AND JIN, H. *Column Stores*. Springer New York, 2018, pp. 518–523.
- [37] ZOMMIOMMY. `mem_dbg-rs`: A trait and derive to recursively compute and print the memory usage of a data structure in rust. https://github.com/zommioomy/mem_dbg-rs, 2023. Accessed: 2025-09-17.